
Site-to-Site VPN on AWS

Virtual Private Gateway + Customer Gateway + Libreswan (IPsec)

A complete practical guide, step by step, in easy English. Every step explains **What** we did, **Why** we did it and **How** to do it, with screenshots, short theory, testing, troubleshooting and a real-life demo.

Part	Where	Role in this lab
AWS Cloud	Mumbai (ap-south-1)	Real AWS network with the Private EC2 (company data server)
Data Center	N. Virginia (us-east-1)	Our pretend office / data centre with the VPN EC2 (Libreswan)
Connection	Over the internet	Encrypted IPsec tunnel between the two

Important: The downloaded VPN configuration file contains secret Pre-Shared Keys (PSK). In this guide the keys are replaced with <YOUR-PSK>. Never share the real keys or upload them to the internet.

Contents

1. Project Overview	4
1.1 What is a Site-to-Site VPN?	4
1.2 Real-life company scenario	4
1.3 What we built in this lab	4
1.4 Topology and IP addresses	5
1.5 Words you must know first	6
2. AWS Setup (What / Why / How)	7
Step 1: Open the VPC service in Mumbai	7
Step 2: Create the VPC "Aws-vpc"	7
Step 3: Create the private subnet	8
Step 4: Check the internet gateway (we do NOT attach one)	9
Step 5: Route table "aws-rt" and subnet association	10
Step 6: Launch the Private EC2 (EC2-A)	11
Step 7: Check EC2-A and note its private IP	13
Step 8: Switch to N. Virginia	13
Step 9: Create "DC-VPC" (192.168.0.0/16)	14
Step 10: Create and attach the internet gateway "DC-IGW"	14
Step 11: Create the two subnets	16
Step 12: Public route table "DC-PUB-ROUTETABLE"	18
Step 13: Private route table "DC-PVT-RT"	20
Step 14: Launch the VPN EC2 "vpn-site-to-site"	21
Step 15: Launch EC2-B (office PC, optional)	23
Step 16: Switch back to Mumbai and open the VPN menu	25
Step 17: Create the Virtual Private Gateway "AWS-VGW"	26
Step 18: Attach the VGW to Aws-vpc	26
Step 19: Add the route 192.168.0.0/16 to the VGW	27
Step 20: Create the Customer Gateway "DC-CGW"	28
Step 21: Create the Site-to-Site VPN connection	29
Step 22: Download the configuration file	31
3. IPsec VPN Configuration (Libreswan)	33
Step 23: Log in to the VPN EC2 from Windows	33
Step 24: Update the server and install Libreswan	35
Step 25: Read the downloaded configuration file	37
Step 26: Apply the configuration on the VPN EC2	40
Step 27: Check that the tunnel is UP	41

4. Routing	43
4.1 AWS route table (aws-rt)	43
4.2 What happens at the Private EC2 (10.0.0.164)	43
4.3 Data centre side	43
4.4 Tunnel vs normal routing	43
5. Testing & Troubleshooting	44
5.1 Test 1: Is the tunnel up?	44
5.2 Test 2: First ping fails (wrong IP)	44
5.3 Test 3: Correct IP but still failing (Security Group)	44
5.4 Deeper checks (if something still fails)	45
5.5 Mistake, reason, solution, final result	45
6. SSH Connectivity	46
6.1 Windows to VPN EC2	46
6.2 VPN EC2 to Private EC2	46
6.3 Nested SSH confusion explained	47
7. Real-Life Practical Demo	48
7.1 Create dummy company data on the Private EC2	48
7.2 Read the files from the VPN EC2 through the VPN	48
7.3 Copy the files with scp	48
7.4 "Permission denied" while copying: reason and solution	49
7.5 Prove the data really went through the VPN	49
8. Important Theory (short and simple)	50
8.1 Site-to-Site vs Remote-Access VPN	50
8.2 IPsec	50
8.3 IKE	50
8.4 ESP and encryption	50
8.5 Other words	50
9. Final Working Architecture	51
Traffic flow when the VPN EC2 pings or copies a file from 10.0.0.164	51
10. Commands Cheat Sheet	52
11. Interview Explanation	53

1. Project Overview

1.1 What is a Site-to-Site VPN?

VPN means **Virtual Private Network**. It builds a secret, locked road (called a **tunnel**) across the public internet. Everything that travels inside the tunnel is **encrypted**, so nobody on the internet can read it.

A **Site-to-Site VPN** joins two complete networks (two "sites") together. For example, a company office network and an AWS network. Computers on both sides talk to each other using **private IP addresses**, as if they were in the same building. Users do not install any VPN software on their laptops; the two VPN devices at the ends of the tunnel do all the work.

Simple example: Imagine two offices in two different cities. You do not want to build your own road between them, so you use the public highway, but you travel in a locked armoured truck. The highway = internet. The armoured truck = VPN tunnel.

1.2 Real-life company scenario

A company called **ABC Pvt Ltd** has an office with a data centre. It also keeps an important **private data server** in AWS (no public IP, so nobody on the internet can touch it). Office computers must read and copy files from this private server every day.

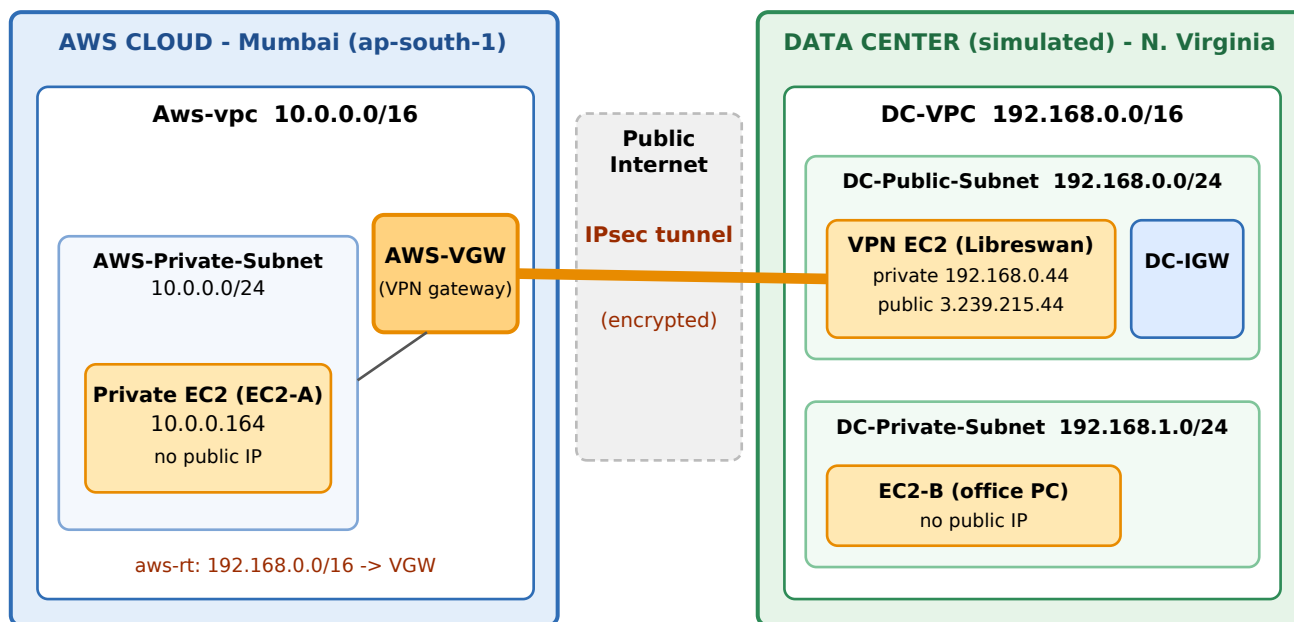
- Sending that data over the plain internet is **not safe**.
- A private fibre line (AWS Direct Connect) is **costly** and can take weeks to set up.
- A **Site-to-Site VPN** is quick (minutes), cheap, and fully encrypted. So the company chooses VPN.

1.3 What we built in this lab

We do not own a real data centre, so we **pretend** one. We create a second network in another AWS region (N. Virginia) and call it the "Data Center". An EC2 server running **Libreswan** acts like the office VPN router. On the AWS side (Mumbai) AWS provides a VPN gateway for us.

Real life	In our lab
Company data centre / office network	DC-VPC (192.168.0.0/16) in N. Virginia
Office VPN router / firewall	VPN EC2 "vpn-site-to-site" running Libreswan
Describing the office router to AWS	Customer Gateway (DC-CGW)
AWS side VPN door	Virtual Private Gateway (AWS-VGW)
Private company server in AWS	Private EC2 (EC2-A) at 10.0.0.164
The secure road between them	IPsec VPN tunnel (AWS-to-DC-VPN)

1.4 Topology and IP addresses



Picture 1: Lab topology. The orange line is the encrypted tunnel.

Item	Name / Value	Details
AWS VPC (Mumbai)	Aws-vpc	10.0.0.0/16
AWS private subnet	AWS-Private-Subnet	10.0.0.0/24, zone ap-south-1a
Private EC2	EC2-A	10.0.0.164, no public IP, key pair new_key
AWS route table	aws-rt	10.0.0.0/16 local, 192.168.0.0/16 to VGW
Virtual Private Gateway	AWS-VGW	vgw-0f4b7e25c8ff34af0, Amazon ASN 64512
DC VPC (N. Virginia)	DC-VPC	192.168.0.0/16
DC public subnet	DC-Public-Subnet	192.168.0.0/24
DC private subnet	DC-Private-Subnet	192.168.1.0/24
VPN EC2	vpn-site-to-site	private 192.168.0.44, public 3.239.215.44, key pair ec2-lab
Extra office PC	EC2-B	In DC-Private-Subnet, no public IP
Customer Gateway	DC-CGW	cgw-06517795899eee4a7, BGP ASN 65000, IP 3.239.215.44
VPN connection	AWS-to-DC-VPN	vpn-03a35f7925fdde83f, static route 192.168.0.0/16
AWS tunnel endpoints	Tunnel 1 / Tunnel 2	13.201.140.220 / 65.0.172.148

Important: The two networks must **not overlap**. AWS uses 10.0.0.0/16 and the data centre uses 192.168.0.0/16. If both used the same range, the VPN would not know where to send packets.

1.5 Words you must know first

Word	Easy meaning
VPC	Your own private network inside AWS.
Subnet	A smaller part of a VPC. Servers are placed inside a subnet.
CIDR (10.0.0.0/16)	A way to write an IP range. /16 = 65,536 addresses, /24 = 256 addresses.
Internet Gateway (IGW)	The door that connects a VPC to the internet.
Route table	A list of rules that says "to reach this network, go through that door".
Security Group (SG)	A firewall for one EC2 server. It allows only the traffic you open.
VGW	Virtual Private Gateway. The AWS end of the VPN tunnel.
CGW	Customer Gateway. A record in AWS that describes <i>your</i> end of the tunnel (its public IP).
Libreswan	Free software that makes a Linux server speak IPsec VPN.
PSK	Pre-Shared Key. A secret password both ends know, to prove who they are.

2. AWS Setup (What / Why / How)

We work in three parts. **Part A:** the AWS side in Mumbai. **Part B:** the pretend data centre in N. Virginia. **Part C:** the VPN objects (VGW, CGW, VPN connection) in Mumbai.

Part A: AWS side (Mumbai, ap-south-1)

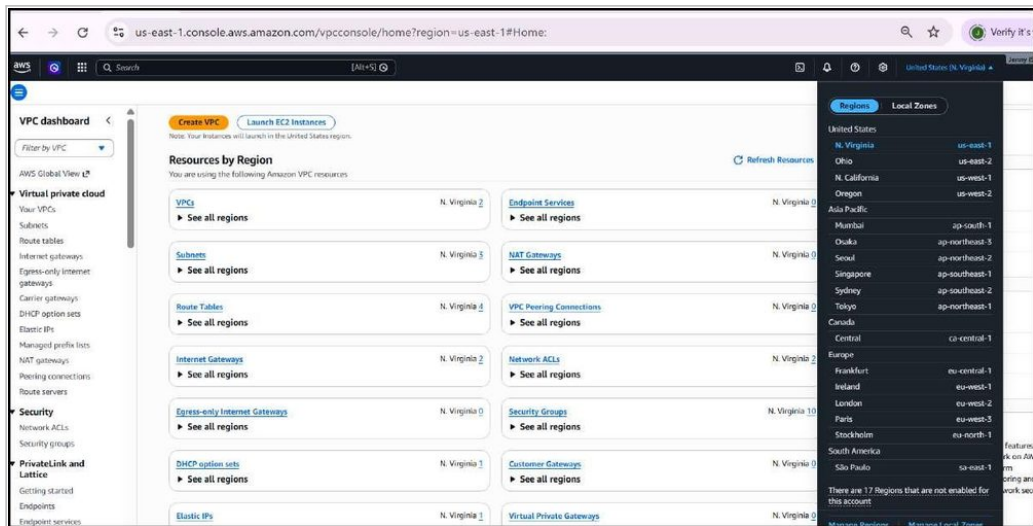
Step 1: Open the VPC service in Mumbai

WHAT Open the VPC console and choose the region **Asia Pacific (Mumbai) ap-south-1**.

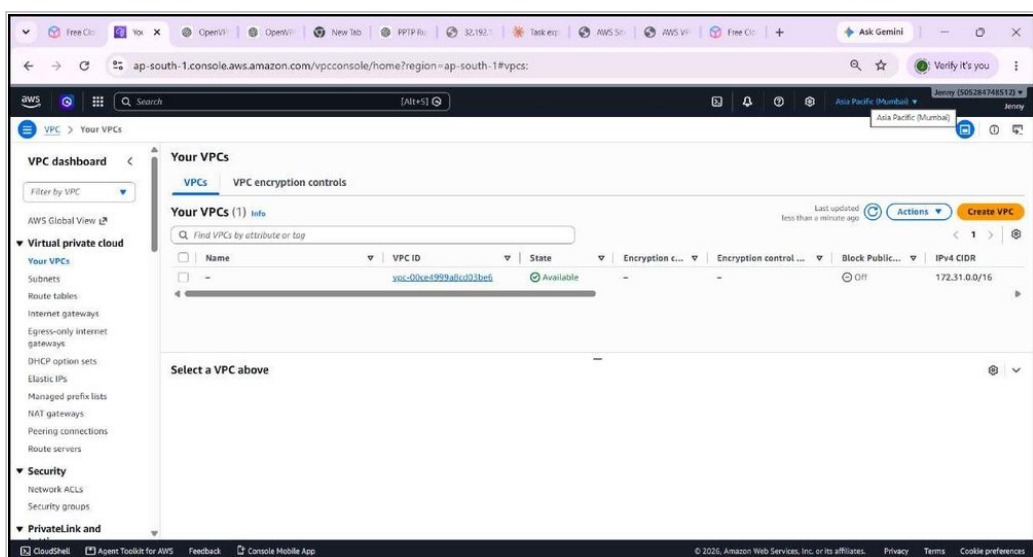
WHY Every AWS resource lives in one region. The AWS cloud part of this lab must be in Mumbai. If the region is wrong, the VGW and VPC cannot be linked.

HOW

1. Sign in to the AWS console and search for **VPC**.
2. Click the region name at the top right and select **Mumbai (ap-south-1)**.
3. In the left menu click **Your VPCs**. Only the default VPC (172.31.0.0/16) is there.



Region list. Pick Mumbai (ap-south-1).



Mumbai region: only the default VPC exists.

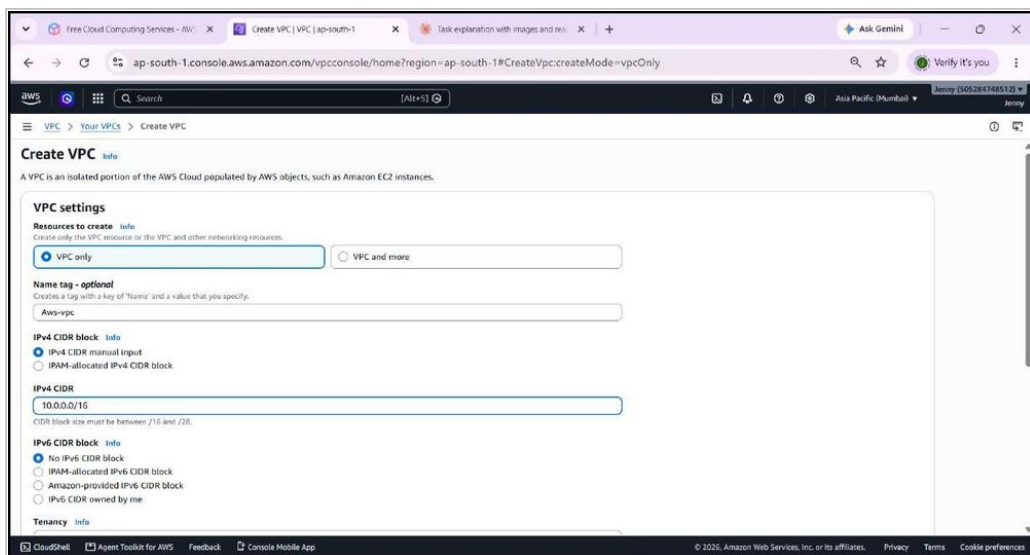
Step 2: Create the VPC "Aws-vpc"

WHAT Create a new private network with the range **10.0.0.0/16**.

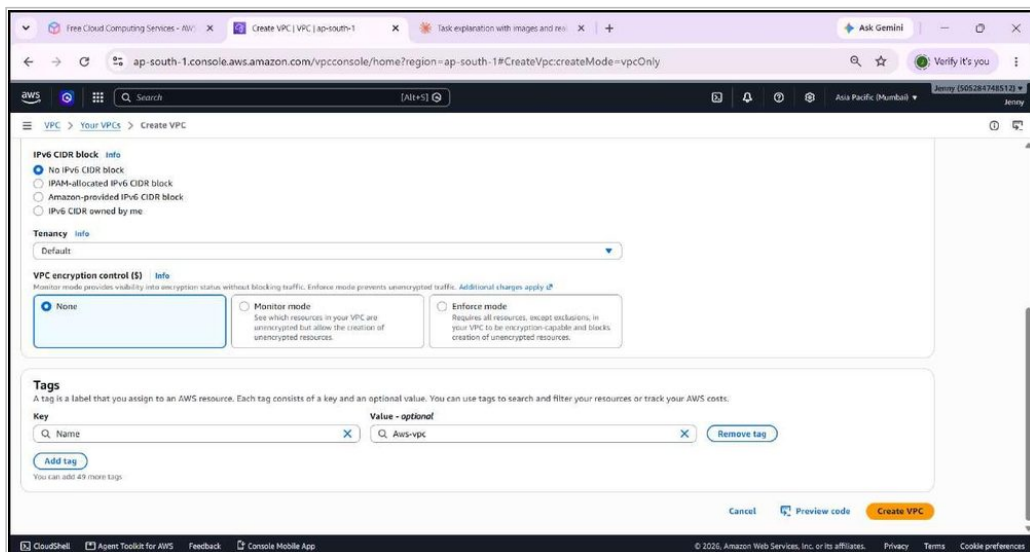
WHY This is our AWS cloud network. The range must be different from the data centre range (192.168.0.0/16).

HOW

1. Click **Create VPC**. Choose **VPC only**.
2. Name tag: **Aws-vpc**.
3. IPv4 CIDR block: **IPv4 CIDR manual input**, value **10.0.0.0/16**.
4. IPv6: No IPv6 CIDR block. Tenancy: Default.
5. Click **Create VPC**.



Create VPC: name *Aws-vpc* and CIDR *10.0.0.0/16*.



Tags section. Name = *Aws-vpc*. Click *Create VPC*.

Step 3: Create the private subnet

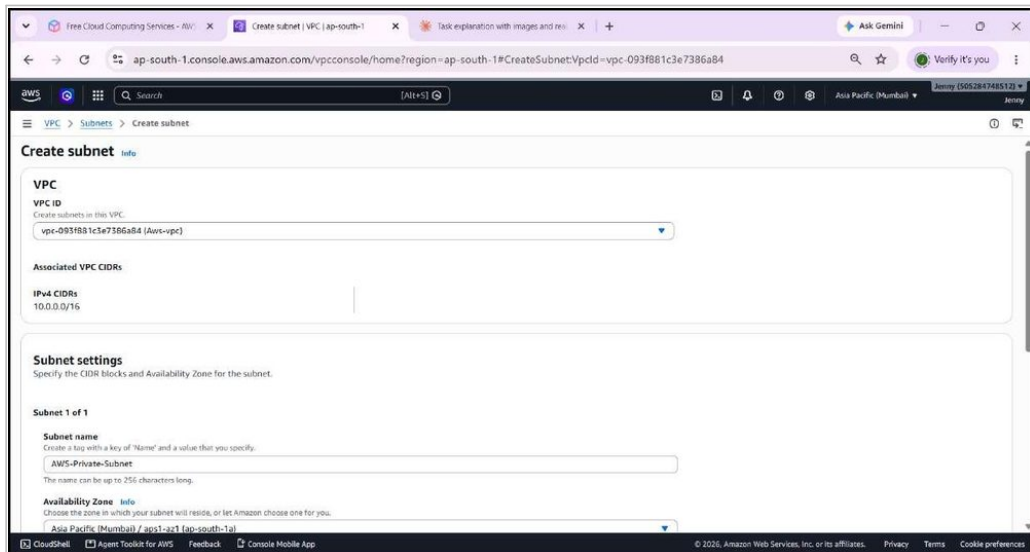
WHAT Create a subnet **AWS-Private-Subnet** with range **10.0.0.0/24** in zone **ap-south-1a**.

WHY Servers are placed in subnets, not directly in the VPC. It is "private" because it has no route to an internet gateway.

HOW

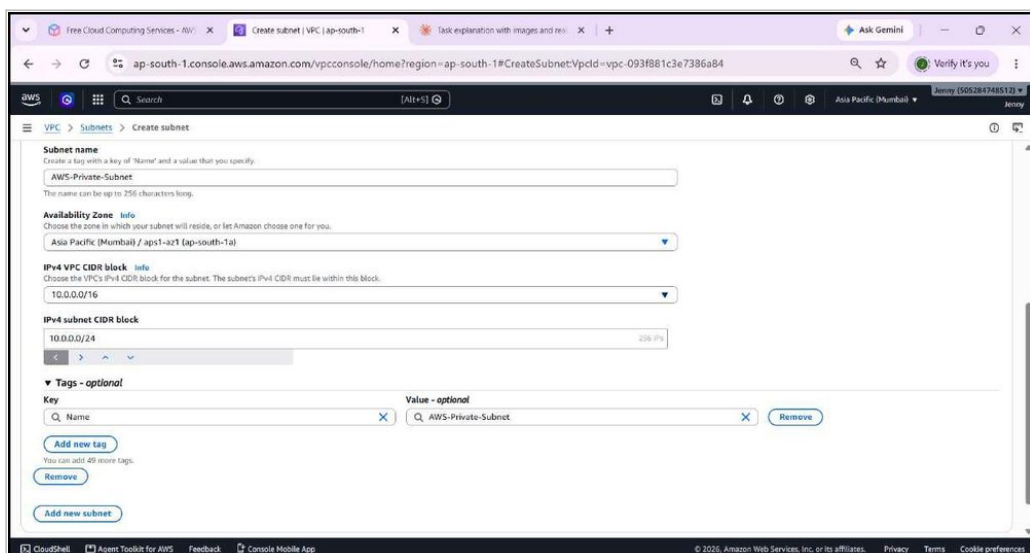
1. Left menu: **Subnets** then **Create subnet**.
2. VPC ID: choose **Aws-vpc**.

3. Subnet name: **AWS-Private-Subnet**. Availability Zone: **ap-south-1a**.
4. IPv4 subnet CIDR block: **10.0.0.0/24**.
5. Click **Create subnet**.



The screenshot shows the 'Create subnet' page in the AWS Management Console. The 'VPC' section is expanded, showing the selected VPC ID 'vpc-093f8b1c3e7386a84 (Aws-vpc)'. The 'Subnet settings' section is also expanded, showing the 'Subnet name' field with the value 'AWS-Private-Subnet' and the 'Availability Zone' dropdown set to 'Asia Pacific (Mumbai) / ap-south-1a'.

Create subnet: select VPC Aws-vpc and give the name.



The screenshot shows the 'Create subnet' page with the 'Subnet name' field set to 'AWS-Private-Subnet' and the 'Availability Zone' dropdown set to 'Asia Pacific (Mumbai) / ap-south-1a'. The 'IPv4 VPC CIDR block' is set to '10.0.0.0/16' and the 'IPv4 subnet CIDR block' is set to '10.0.0.0/24'. The 'Tags' section is expanded, showing a tag with key 'Name' and value 'AWS-Private-Subnet'.

Subnet CIDR 10.0.0.0/24 and the Name tag.

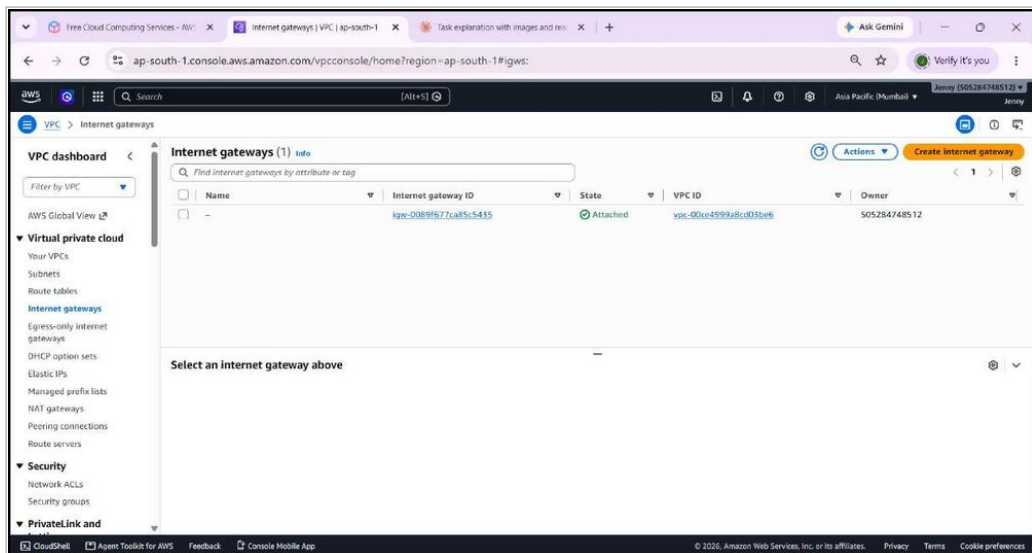
Step 4: Check the internet gateway (we do NOT attach one)

WHAT Open **Internet gateways** and look at the list.

WHY We want the AWS network to be **fully private**. Only the default VPC has an internet gateway. Our Aws-vpc has none, so the only way in is the VPN.

HOW

1. Left menu: **Internet gateways**.
2. You see one gateway, attached to the default VPC only.
3. Do not create or attach any gateway to Aws-vpc.



The only internet gateway belongs to the default VPC, not Aws-vpc.

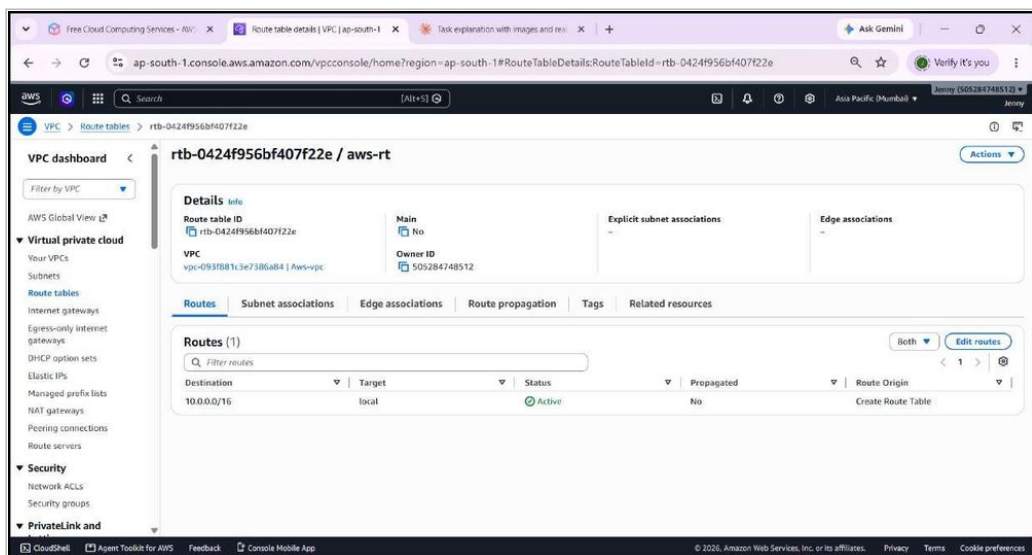
Step 5: Route table "aws-rt" and subnet association

WHAT Use a custom route table named **aws-rt** for Aws-vpc and connect (associate) it with AWS-Private-Subnet.

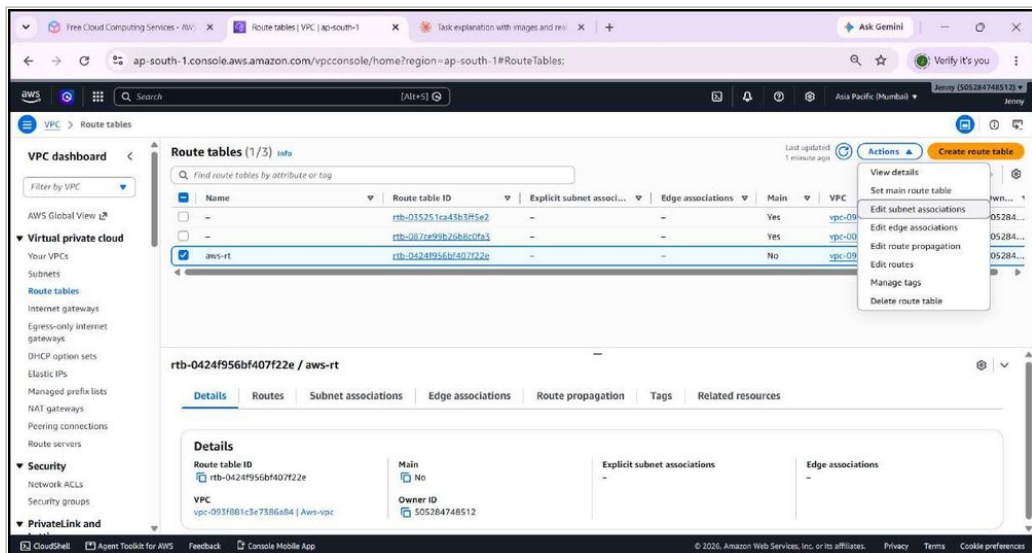
WHY A subnet follows one route table. A custom table lets us add the VPN route later without touching other tables.

HOW

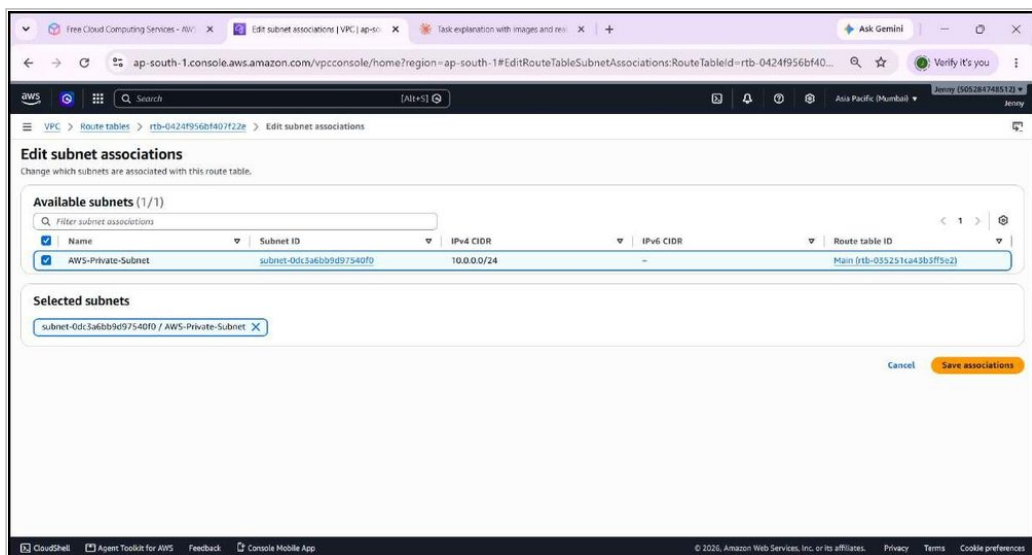
1. Left menu: **Route tables** then **Create route table** (name **aws-rt**, VPC **Aws-vpc**).
2. Tick **aws-rt**, click **Actions** then **Edit subnet associations**.
3. Tick **AWS-Private-Subnet** and click **Save associations**.
4. At this moment the table has only one route: **10.0.0.0/16 local**.



aws-rt has only the local route for now.



Actions menu, then Edit subnet associations.



Select AWS-Private-Subnet and save.

Step 6: Launch the Private EC2 (EC2-A)

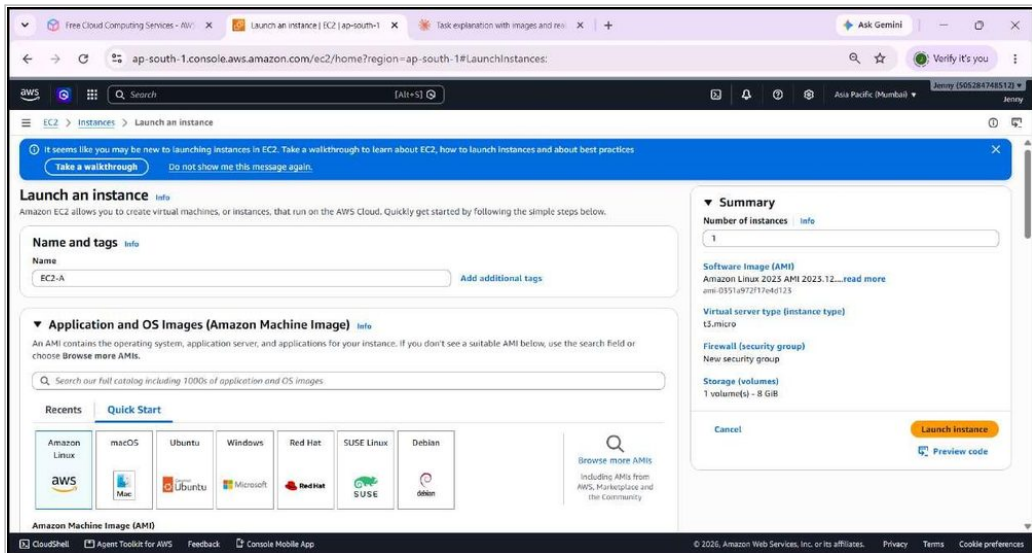
WHAT Launch a **t3.micro** Amazon Linux 2023 server in AWS-Private-Subnet with **no public IP**.

WHY This is our "company private data server". With no public IP and no internet gateway, nobody from the internet can reach it. Only the VPN can.

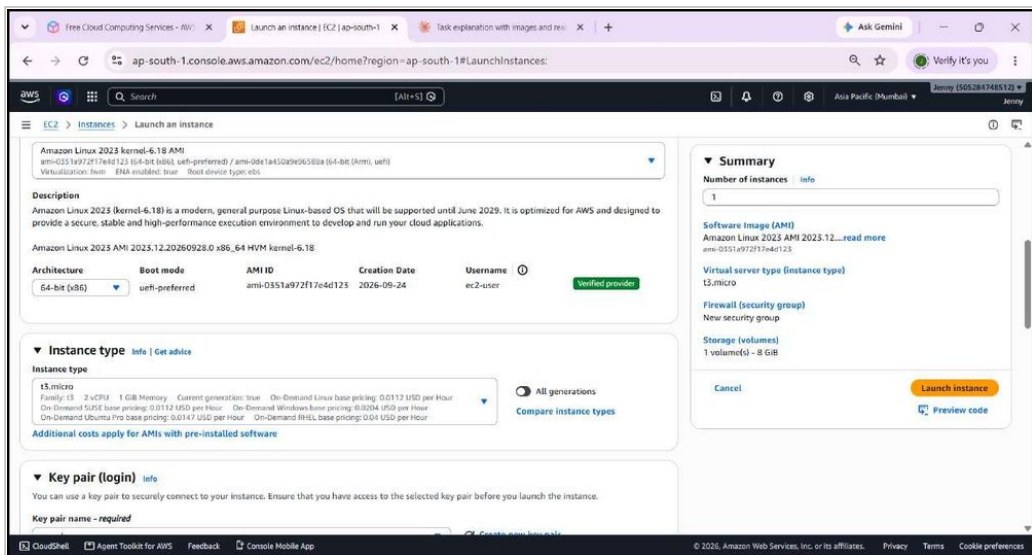
HOW

1. EC2 service, **Launch instance**. Name: **EC2-A**.
2. AMI: **Amazon Linux 2023**. Instance type: **t3.micro**.
3. Key pair: **new_key** (the key pair must exist in Mumbai).
4. Network: VPC **Aws-vpc**, subnet **AWS-Private-Subnet**, Auto-assign public IP: **Disable**.
5. Security group: create new with SSH (port 22). We add ICMP later for ping.
6. Storage 8 GiB gp3. Click **Launch instance**.

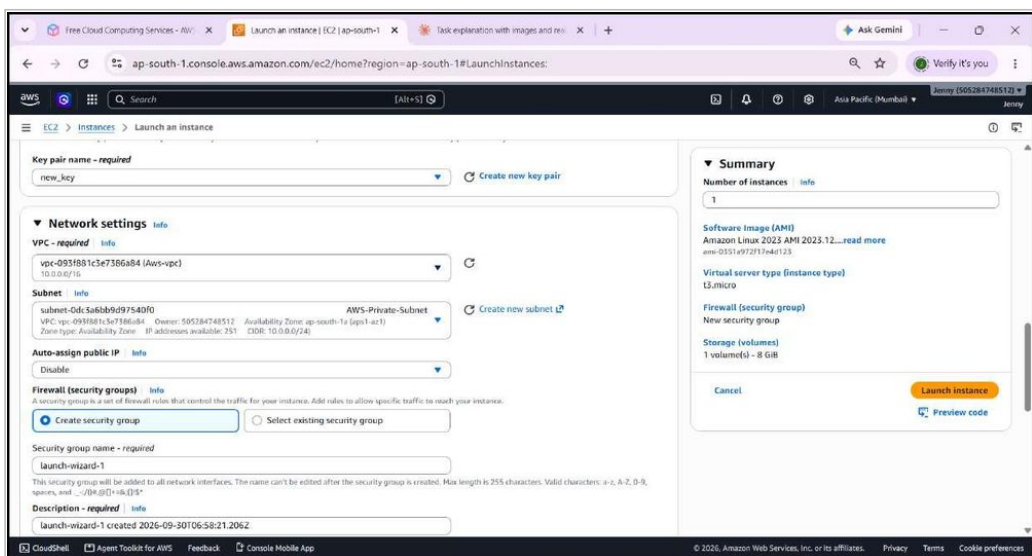
Tip: For SSH, "Anywhere" is only OK for a lab. For better security set the source to **192.168.0.0/16** (the data centre network) so only VPN users can SSH.



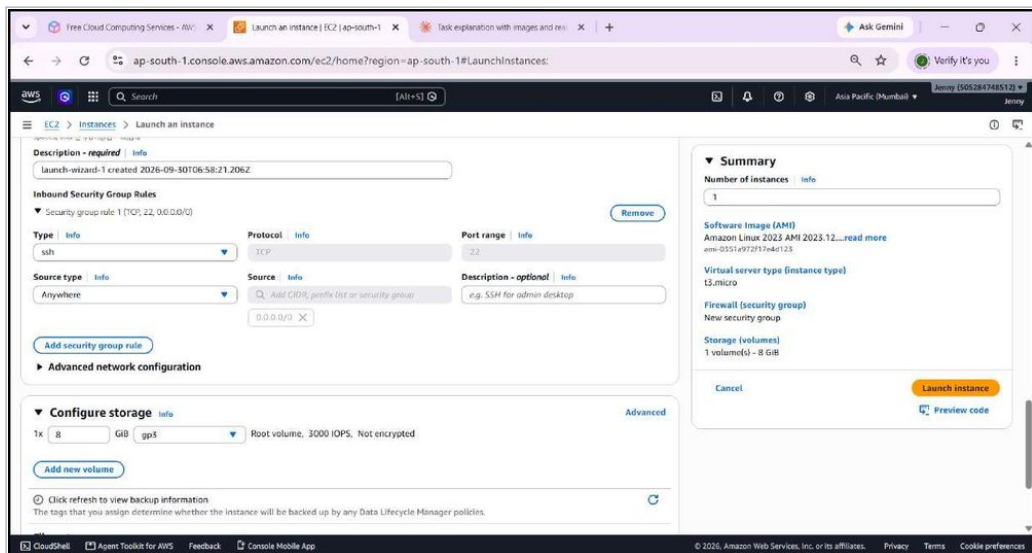
Launch instance: name EC2-A, Amazon Linux.



Instance type t3.micro and the AMI details.



Key pair new_key, network Aws-vpc, auto-assign public IP = Disable.



Security group rule for SSH and 8 GiB storage.

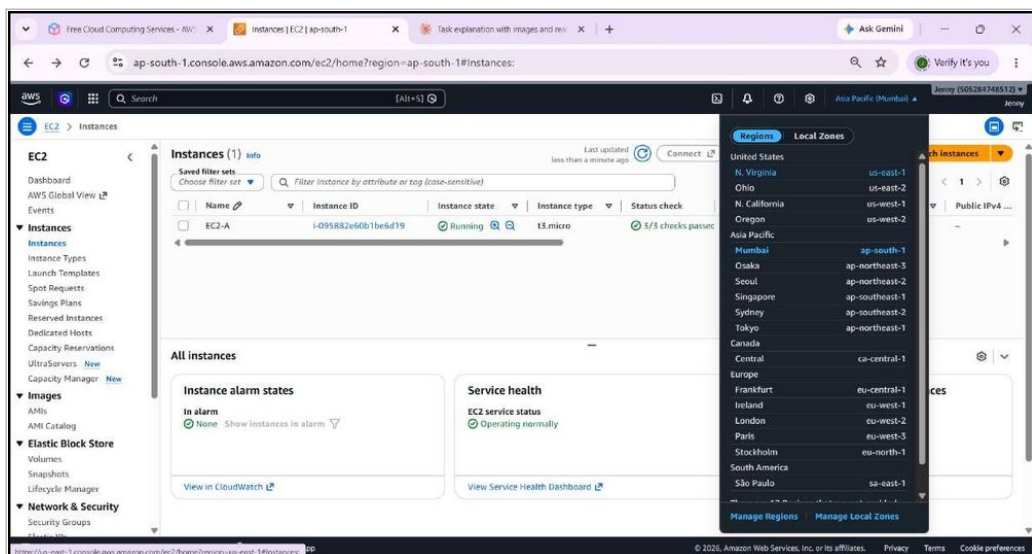
Step 7: Check EC2-A and note its private IP

WHAT Open the instance list and read the **Private IPv4 address** of EC2-A.

WHY We need the exact IP for ping and SSH tests. In this lab it is **10.0.0.164**. (Later we will see what happens if you guess an IP, like 10.0.0.1.)

HOW

1. EC2, **Instances**. Wait until state is **Running** and 3/3 checks pass.
2. Click the instance and copy the **Private IPv4 address**.



EC2-A running. The region menu is open; next we switch to N. Virginia.

Part B: Data Center side (N. Virginia, us-east-1)

Step 8: Switch to N. Virginia

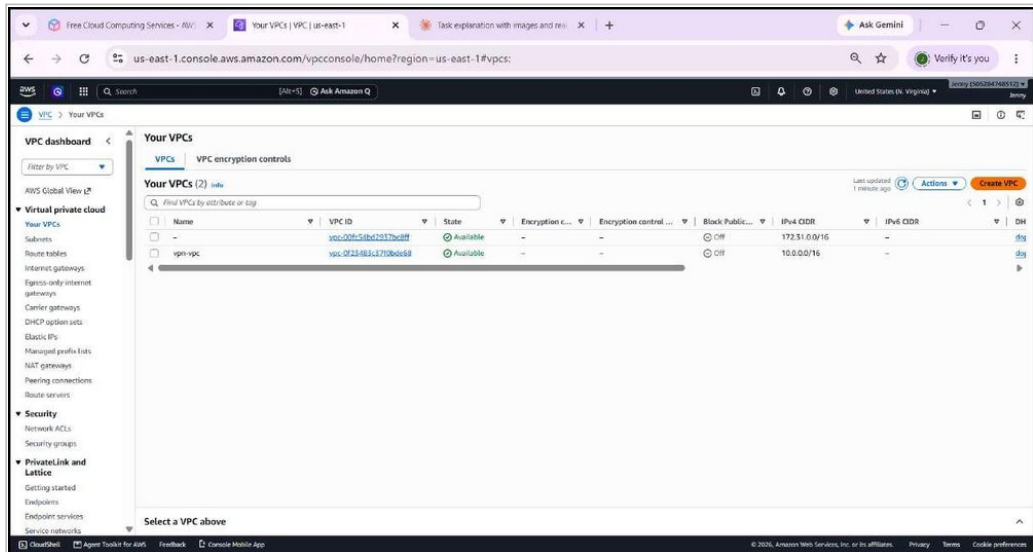
WHAT Change the region to **US East (N. Virginia) us-east-1** and open **Your VPCs**.

WHY The data centre is a different "site", so we place it in a different region.

HOW

1. Click the region name and select **N. Virginia**.
2. Open VPC, then **Your VPCs**.

3. Other VPCs in the list (default and vpn-vpc) belong to earlier labs. Ignore them.



N. Virginia: existing VPCs from earlier labs.

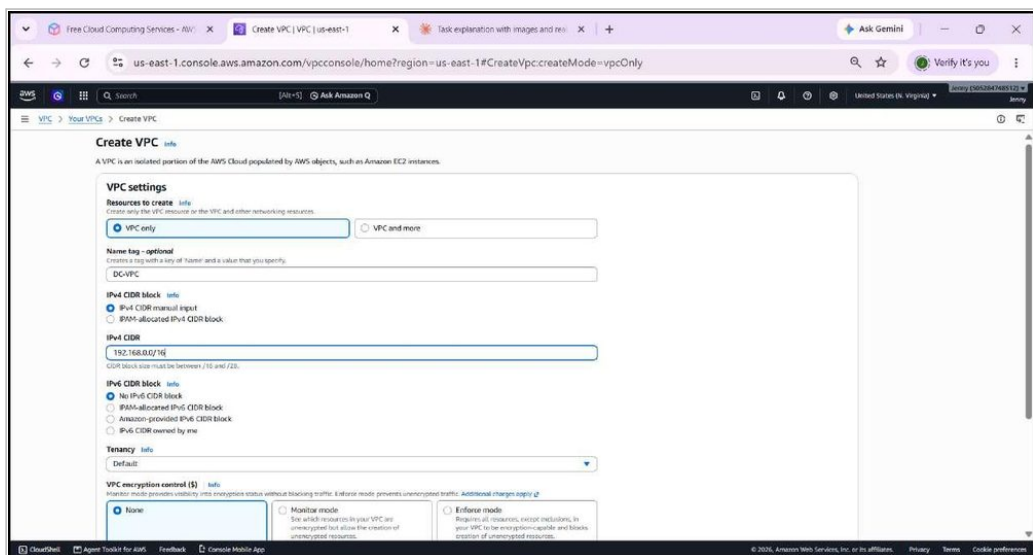
Step 9: Create "DC-VPC" (192.168.0.0/16)

WHAT Create the data centre network.

WHY This network pretends to be the company office. Its range 192.168.0.0/16 is different from AWS (10.0.0.0/16).

HOW

1. **Create VPC**, choose **VPC only**.
2. Name: **DC-VPC**. IPv4 CIDR: **192.168.0.0/16**.
3. Click **Create VPC**.



DC-VPC with CIDR 192.168.0.0/16.

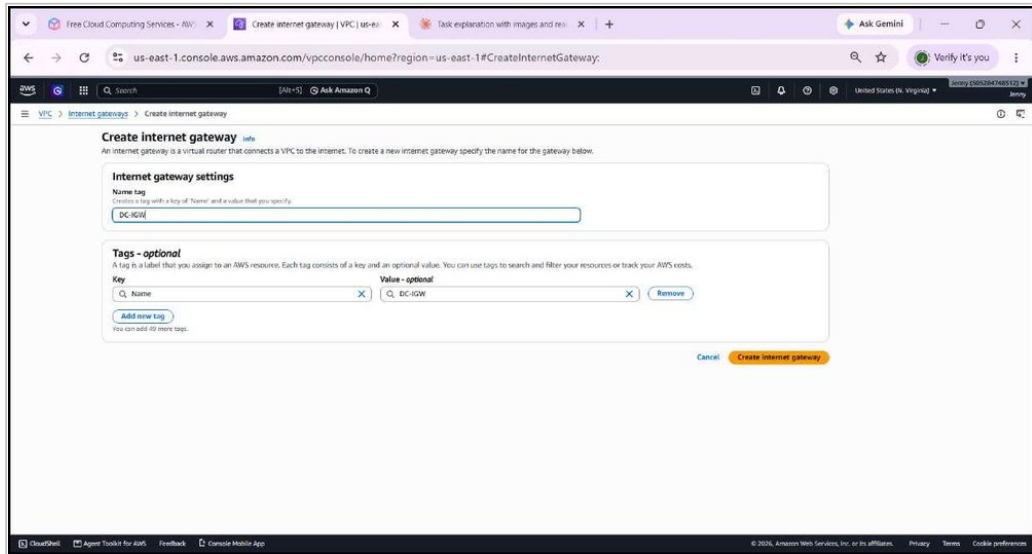
Step 10: Create and attach the internet gateway "DC-IGW"

WHAT Create an internet gateway and attach it to DC-VPC.

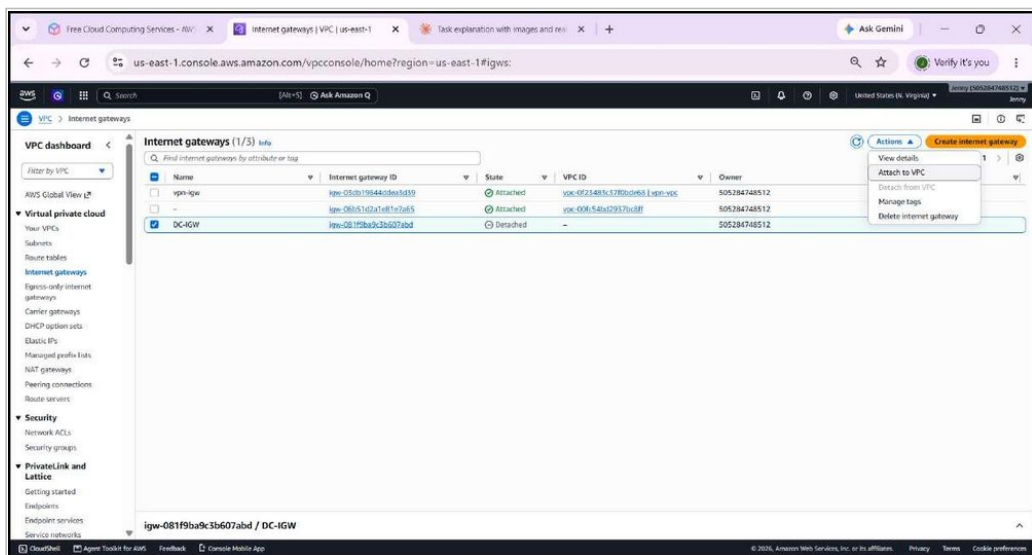
WHY The VPN EC2 needs the internet because the tunnel travels over the internet. Also we need it to log in with SSH.

HOW

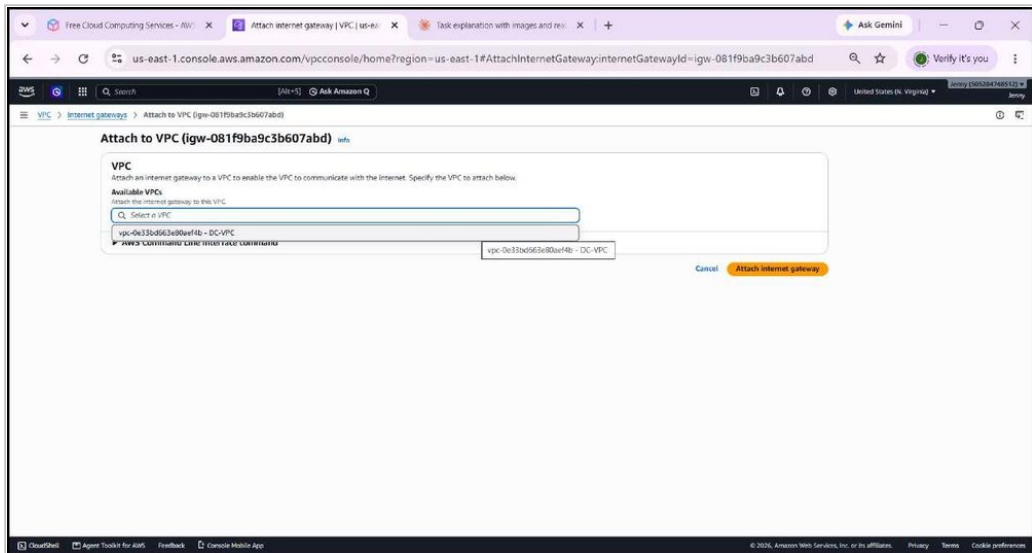
1. **Internet gateways, Create internet gateway, name DC-IGW.**
2. Its state is **Detached**. Select it, **Actions, Attach to VPC**.
3. Choose **DC-VPC** and click **Attach internet gateway**.



Create DC-IGW.



DC-IGW is Detached. Use Actions, Attach to VPC.



Select DC-VPC.

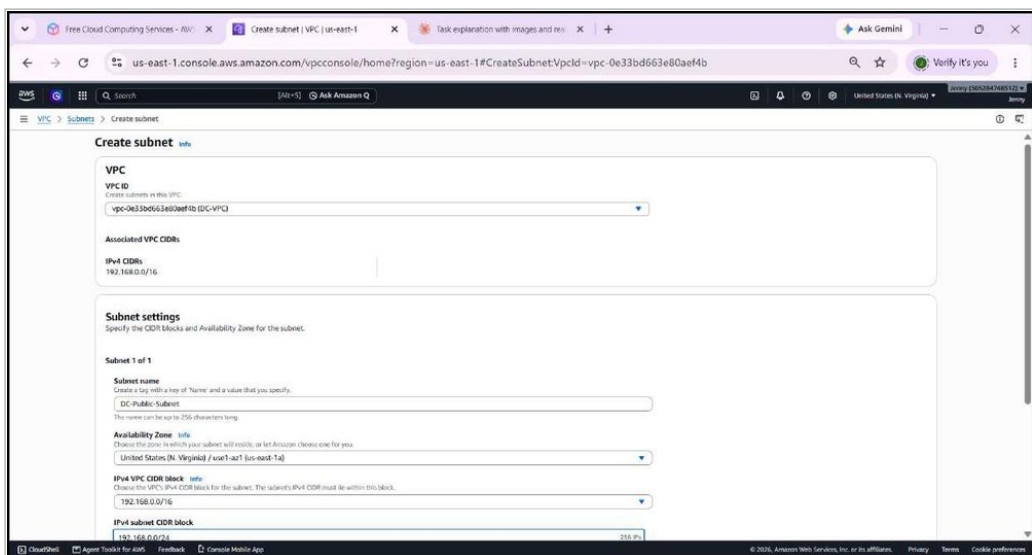
Step 11: Create the two subnets

WHAT Create **DC-Public-Subnet 192.168.0.0/24** and **DC-Private-Subnet 192.168.1.0/24**, both in us-east-1a.

WHY Public subnet = for the VPN server (needs internet). Private subnet = for office computers (no direct internet).

HOW

1. **Subnets, Create subnet, VPC DC-VPC.**
2. Name **DC-Public-Subnet**, zone us-east-1a, CIDR **192.168.0.0/24**.
3. Click **Add new subnet** and add **DC-Private-Subnet**, CIDR **192.168.1.0/24** (or create it again separately).
4. Click **Create subnet**.



DC-Public-Subnet settings.

The screenshot shows the 'Create subnet' page in the AWS Management Console. The page is for creating a new subnet within a VPC. The 'Subnet name' is 'DC-Public-Subnet'. The 'Availability Zone' is 'us-east-1a'. The 'IPv4 VPC CIDR block' is '192.168.0.0/16'. The 'IPv4 subnet CIDR block' is '192.168.0.0/24'. There is a tag with the key 'Name' and value 'DC-Public-Subnet'. The 'Create subnet' button is highlighted in orange.

DC-Public-Subnet CIDR 192.168.0.0/24 and tags.

The screenshot shows the 'Create subnet' page in the AWS Management Console. The page is for creating a new subnet within a VPC. The 'Subnet name' is 'DC-Private-Subnet'. The 'Availability Zone' is 'us-east-1a'. The 'IPv4 VPC CIDR block' is '192.168.0.0/16'. The 'IPv4 subnet CIDR block' is '192.168.1.0/24'. There is a tag with the key 'Name' and value 'DC-Private-Subnet'. The 'Create subnet' button is highlighted in orange.

DC-Private-Subnet settings.

The screenshot shows the 'Create subnet' page in the AWS Management Console. The page is for creating a new subnet within a VPC. The 'Subnet name' is 'DC-Private-Subnet'. The 'Availability Zone' is 'us-east-1a'. The 'IPv4 VPC CIDR block' is '192.168.0.0/16'. The 'IPv4 subnet CIDR block' is '192.168.1.0/24'. There is a tag with the key 'Name' and value 'DC-Private-Subnet'. The 'Create subnet' button is highlighted in orange.

DC-Private-Subnet CIDR 192.168.1.0/24 and tags.

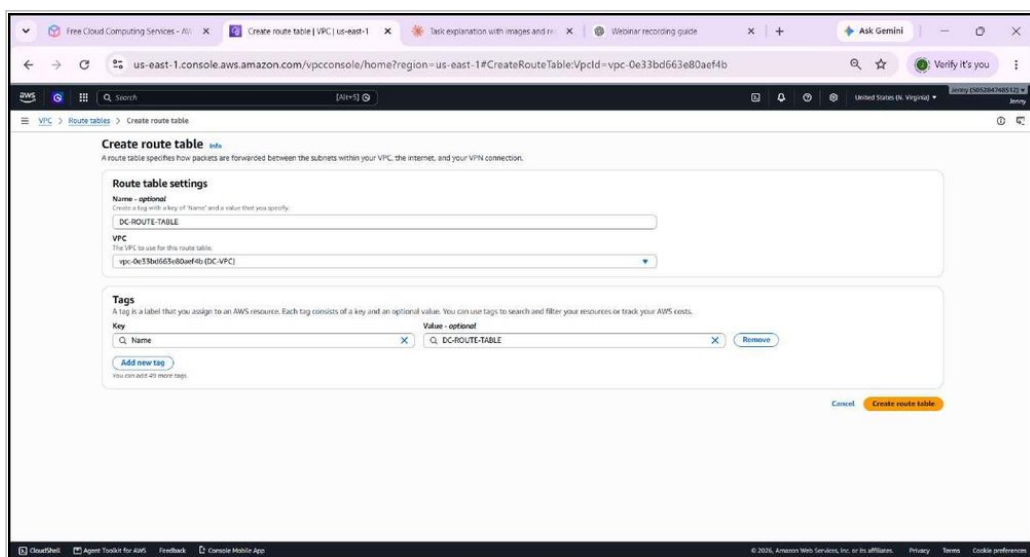
Step 12: Public route table "DC-PUB-ROUTETABLE"

WHAT Create a route table, add a default route **0.0.0.0/0** to **DC-IGW**, and attach it to DC-Public-Subnet.

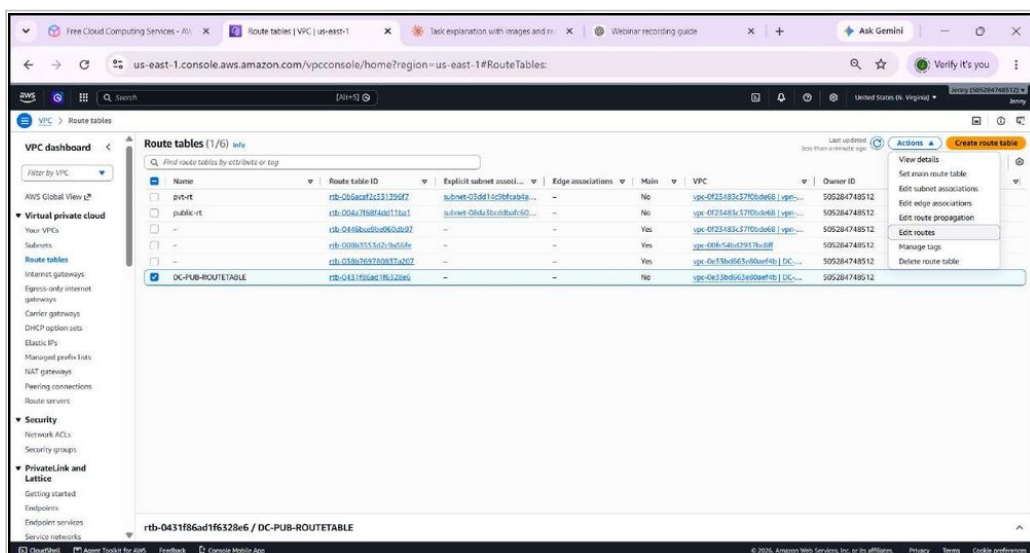
WHY This rule makes the subnet "public": any traffic not for the local network goes out to the internet.

HOW

1. **Route tables, Create route table**, VPC **DC-VPC**. Use the name **DC-PUB-ROUTETABLE**.
2. Select it, **Actions, Edit routes, Add route**.
3. Destination **0.0.0.0/0**, Target **Internet Gateway** then choose **DC-IGW**. Save changes.
4. **Actions, Edit subnet associations**, tick **DC-Public-Subnet**, Save associations.

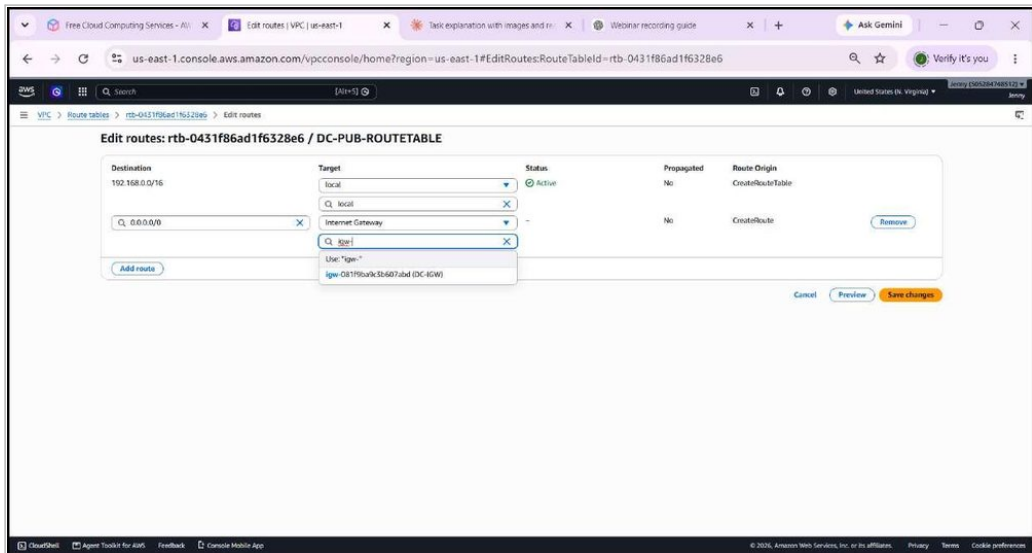


Create the route table (in this picture the name box shows DC-ROUTE-TABLE; use DC-PUB-ROUTETABLE).

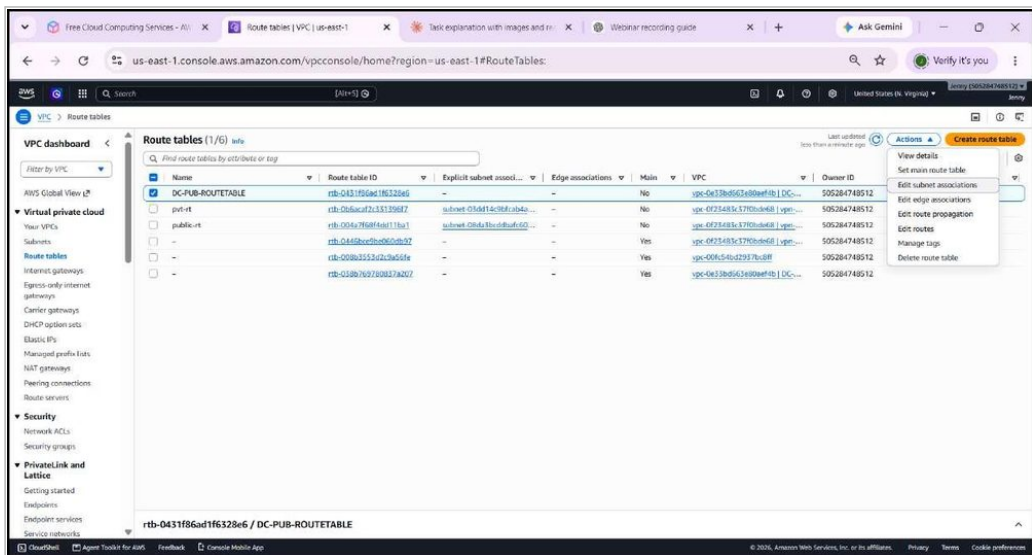


Name	Route table ID	Explicit subnet assoc...	Edge associations	Main	VPC	Owner ID
priv-rt	rtb-0b6acaf1c31396f7	subnet-02d41a3bfc0bda...	-	No	vpc-0f31483c3770b4e68 vpc-	S95284748512
public-rt	rtb-0d4a788f4a4113a1	subnet-0bda3e0c8a0f4d...	-	No	vpc-0f31483c3770b4e68 vpc-	S95284748512
-	rtb-04f8b4d9d9d9d9d9	-	-	Yes	vpc-0f31483c3770b4e68 vpc-	S95284748512
-	rtb-00b0b151c1b05d6	-	-	Yes	vpc-0f31483c3770b4e68 vpc-	S95284748512
-	rtb-01b076a7d9d7a2d2	-	-	Yes	vpc-0e33bd563e80aef4b DC-	S95284748512
DC-PUB-ROUTETABLE	rtb-0431f86ad1f6328e6	-	-	No	vpc-0e33bd563e80aef4b DC-	S95284748512

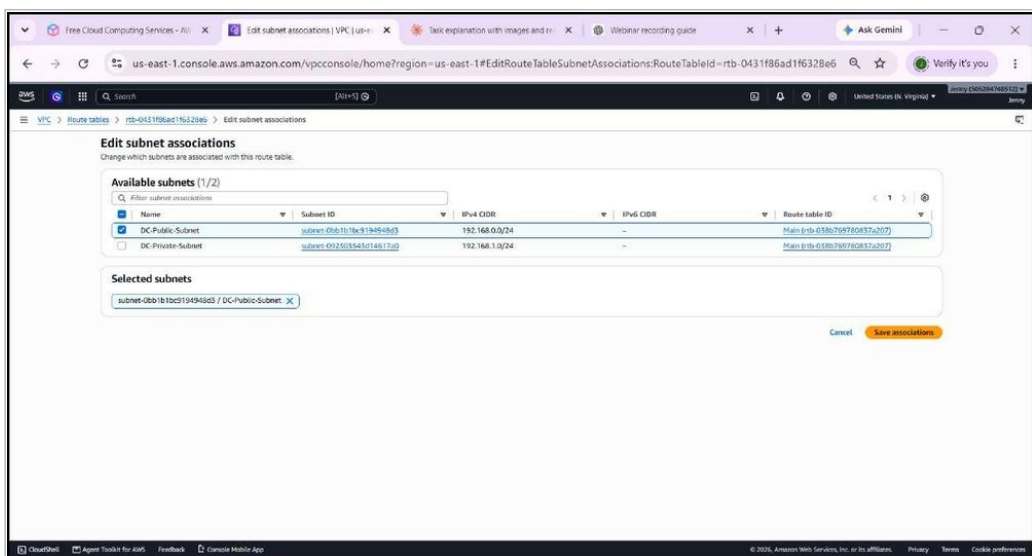
Actions, then Edit routes.



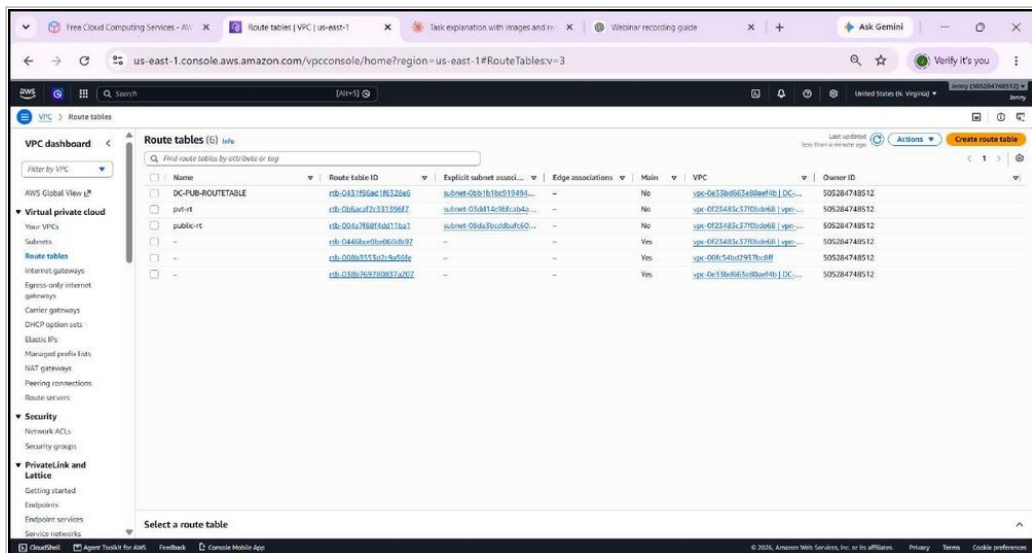
Add 0.0.0.0/0 with target Internet Gateway DC-IGW.



Actions, then Edit subnet associations.



Select DC-Public-Subnet.



Route table list after the changes.

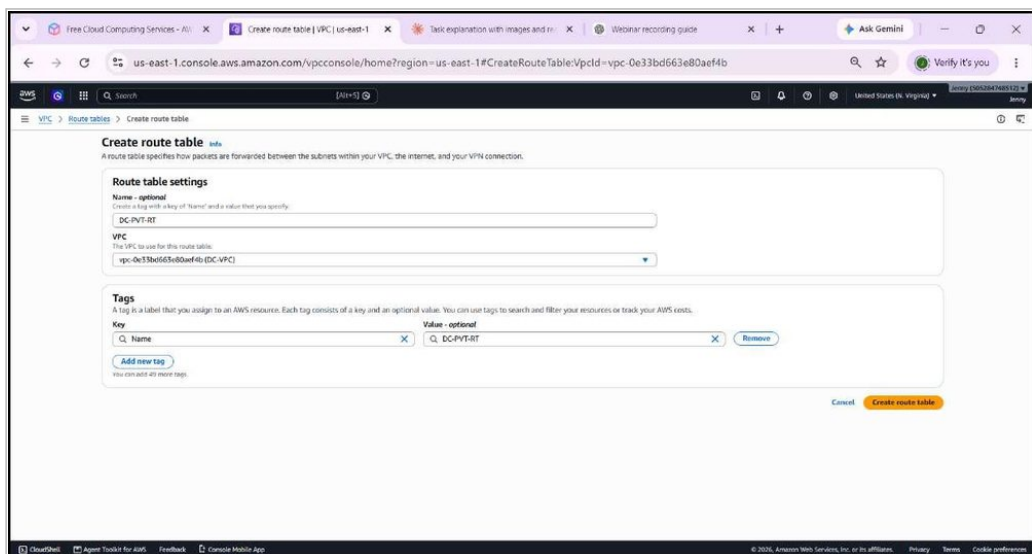
Step 13: Private route table "DC-PVT-RT"

WHAT Create a second route table and attach it to DC-Private-Subnet. Add no internet route.

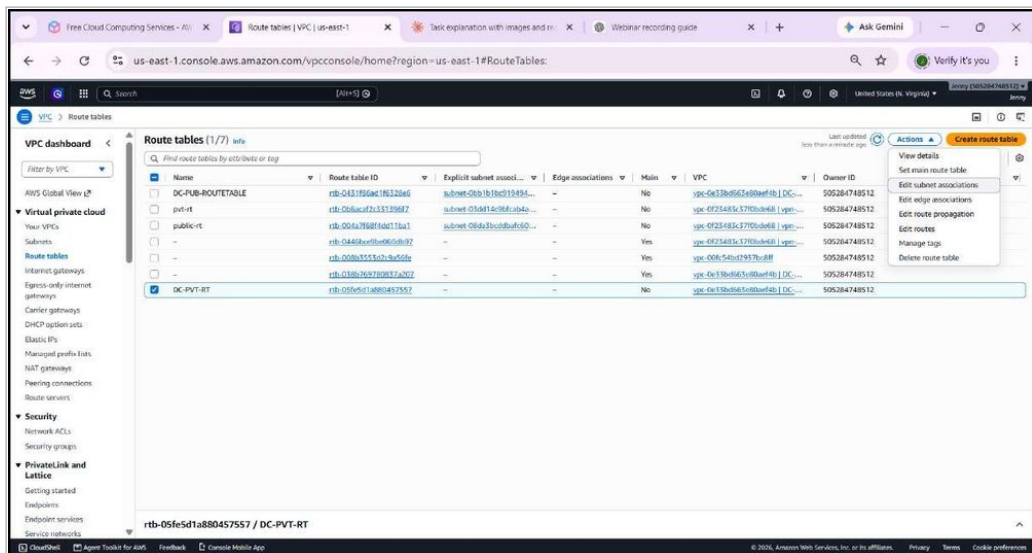
WHY Office computers in the private subnet must not go directly to the internet.

HOW

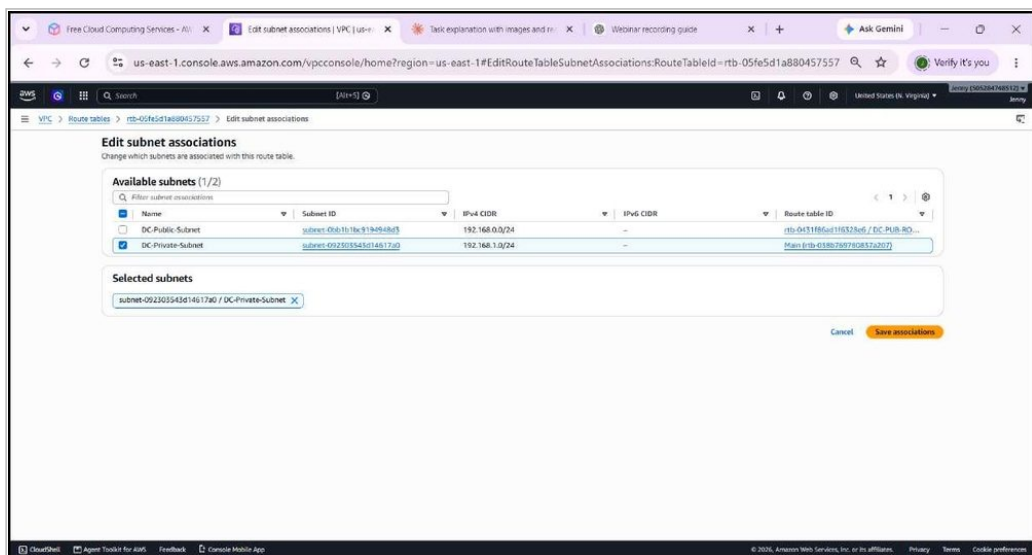
1. **Create route table**, name **DC-PVT-RT**, VPC **DC-VPC**.
2. Select it, **Actions**, **Edit subnet associations**.
3. Tick **DC-Private-Subnet** and save.



Create DC-PVT-RT.



Edit subnet associations.



Select DC-Private-Subnet.

Step 14: Launch the VPN EC2 "vpn-site-to-site"

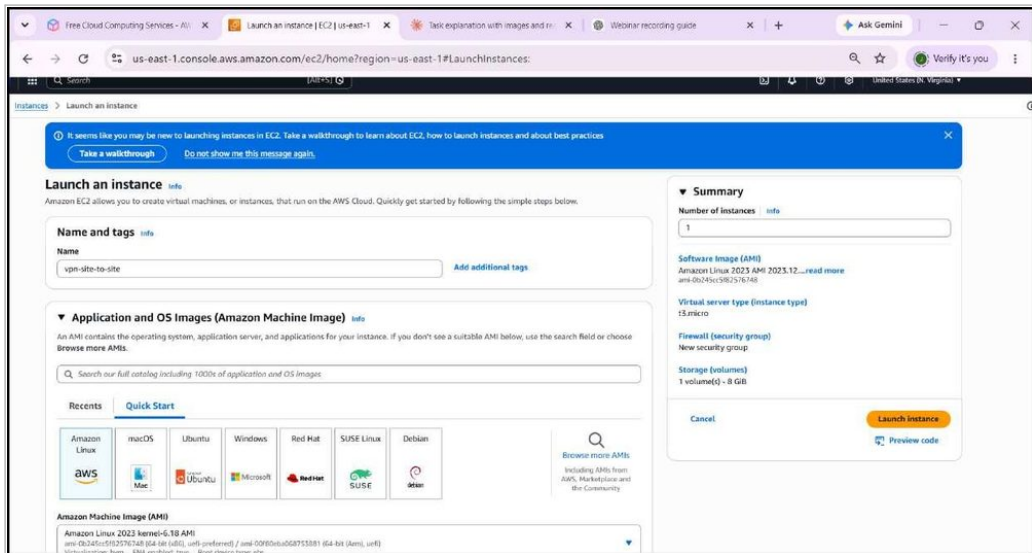
WHAT Launch a t3.micro Amazon Linux 2023 server in **DC-Public-Subnet** with a **public IP**.

WHY This server is our **office VPN router**. It will run Libreswan. It needs a public IP because AWS must know where to connect the tunnel. In this lab the public IP is **3.239.215.44**.

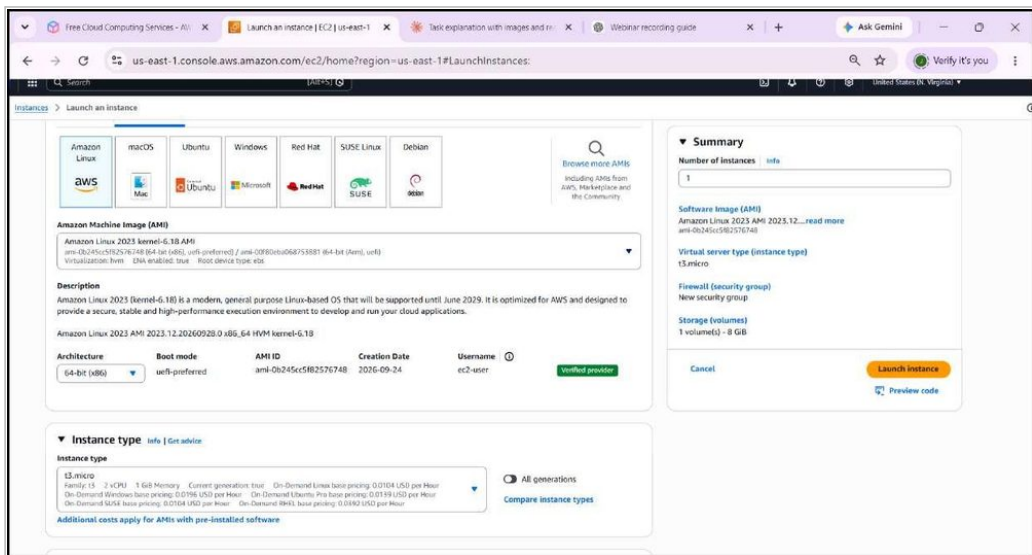
HOW

1. **Launch instance**, name **vpn-site-to-site**.
2. AMI Amazon Linux 2023, type t3.micro.
3. Key pair: **ec2-lab**.
4. Network: VPC **DC-VPC**, subnet **DC-Public-Subnet**, Auto-assign public IP **Enable**.
5. Security group: new, with SSH (22). See the security group table below for extra rules.
6. Launch.

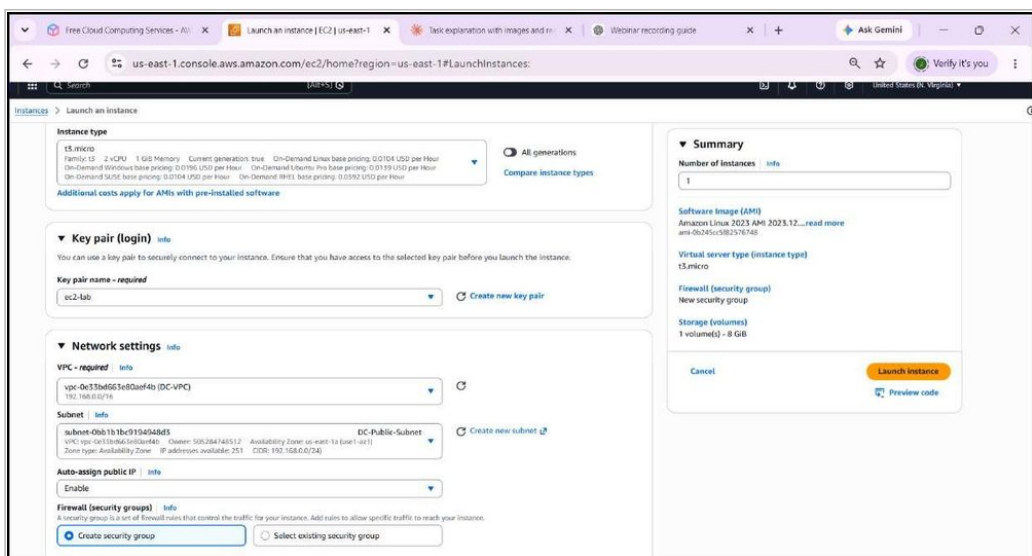
Important: A normal auto-assigned public IP **changes** when the instance is stopped and started. Then the tunnel breaks and you must update the Customer Gateway. In real life attach an **Elastic IP** to the VPN server.



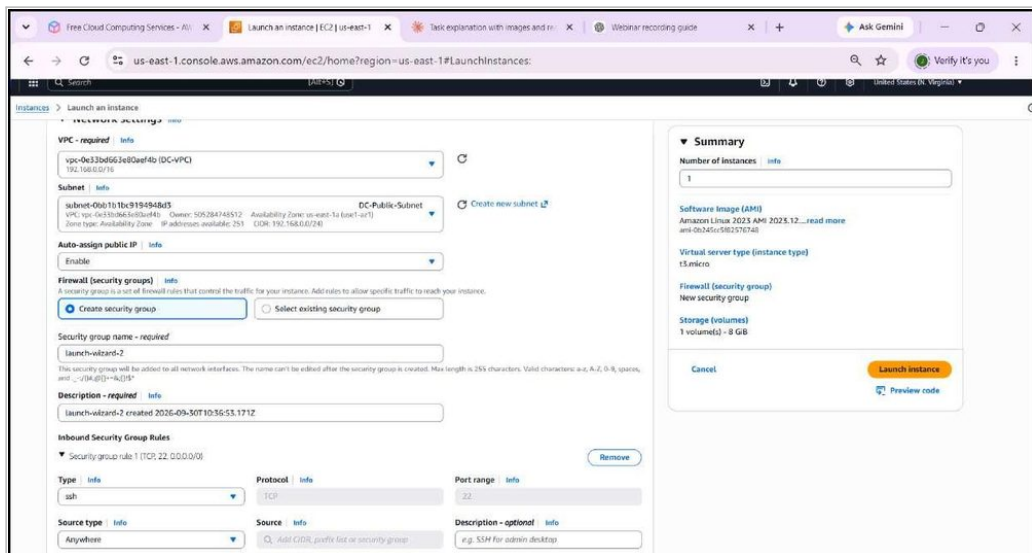
Name the VPN server *vpn-site-to-site*.



Amazon Linux 2023 and *t3.micro*.



Key pair *ec2-lab*, *DC-VPC*, *DC-Public-Subnet*, public IP Enable.



Security group with SSH rule.

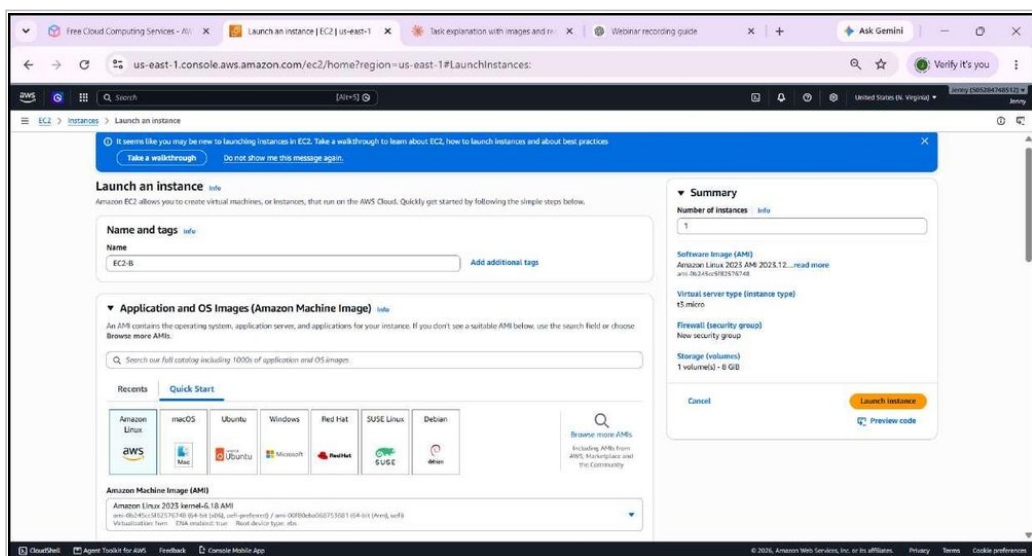
Step 15: Launch EC2-B (office PC, optional)

WHAT Launch another small server in **DC-Private-Subnet** with no public IP.

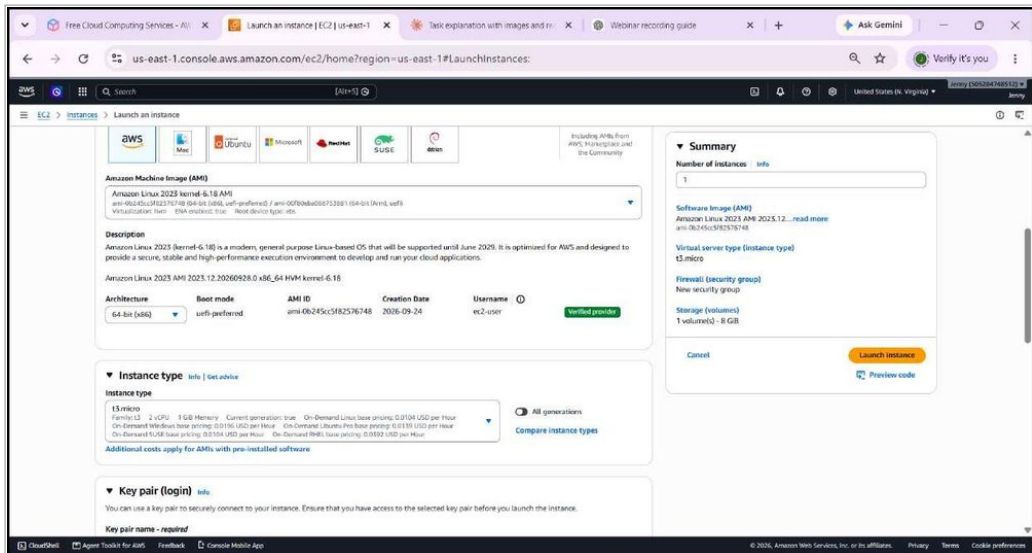
WHY It pretends to be a normal office computer. The main tests in this guide use the VPN EC2, so EC2-B is only for extending the lab later.

HOW

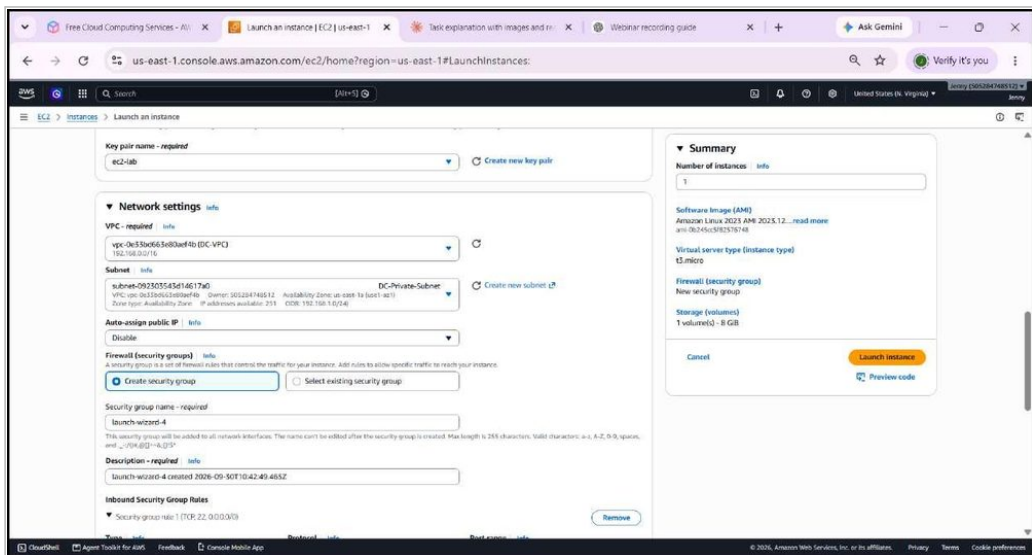
1. **Launch instance**, name **EC2-B**, Amazon Linux 2023, t3.micro.
2. Key pair **ec2-lab**.
3. Network: **DC-VPC**, **DC-Private-Subnet**, Auto-assign public IP **Disable**.
4. Launch.



Name EC2-B.



Same AMI and instance type.



Private subnet, public IP disabled.

Security Groups (firewall rules) summary

A Security Group only allows what you open. Return traffic is allowed automatically. These are the rules this lab needs:

Server	Inbound rule	Source	Why
VPN EC2 (DC)	SSH 22/TCP	Your IP (lab: Anywhere)	To log in from Windows
VPN EC2 (DC)	UDP 500 and UDP 4500	AWS tunnel IPs (or 0.0.0.0/0 for lab)	IKE and NAT-T: the tunnel setup packets
VPN EC2 (DC)	All ICMP-IPv4	10.0.0.0/16	To allow ping replies from AWS side servers
Private EC2 (AWS)	SSH 22/TCP	192.168.0.0/16	SSH through the VPN
Private EC2 (AWS)	All ICMP-IPv4	192.168.0.0/16	Ping through the VPN (this fixed our ping problem)

Route tables summary

Route table	Destination	Target	Meaning
aws-rt (Mumbai)	10.0.0.0/16	local	Talk inside Aws-vpc
aws-rt (Mumbai)	192.168.0.0/16	AWS-VGW	Data centre traffic goes into the VPN
DC-PUB-ROUTETABLE	192.168.0.0/16	local	Talk inside DC-VPC
DC-PUB-ROUTETABLE	0.0.0.0/0	DC-IGW	Internet access (and the tunnel itself)
DC-PVT-RT	192.168.0.0/16	local	Talk inside DC-VPC only

Part C: VPN objects on the AWS side (Mumbai)

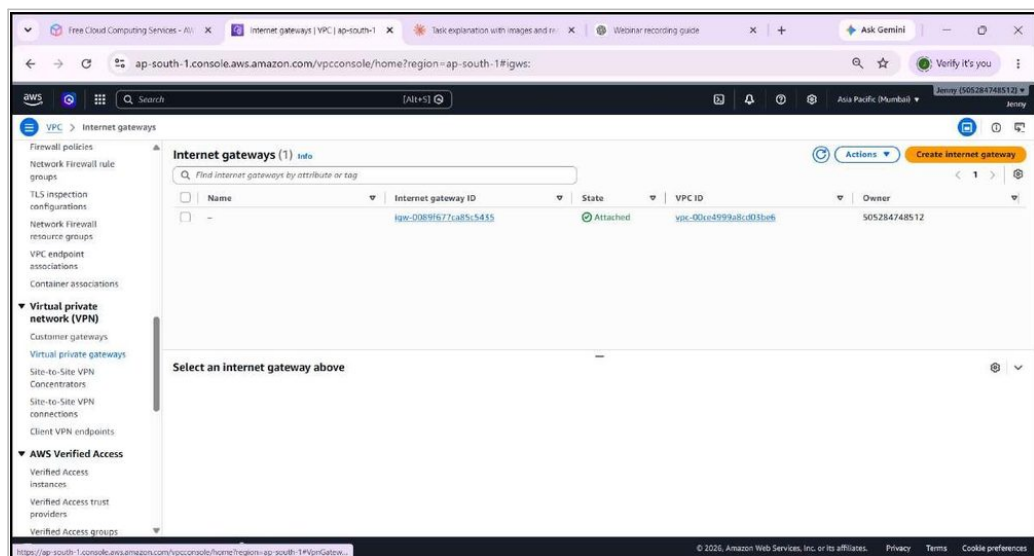
Step 16: Switch back to Mumbai and open the VPN menu

WHAT Go back to region Mumbai. In the VPC left menu scroll down to **Virtual private network (VPN)**.

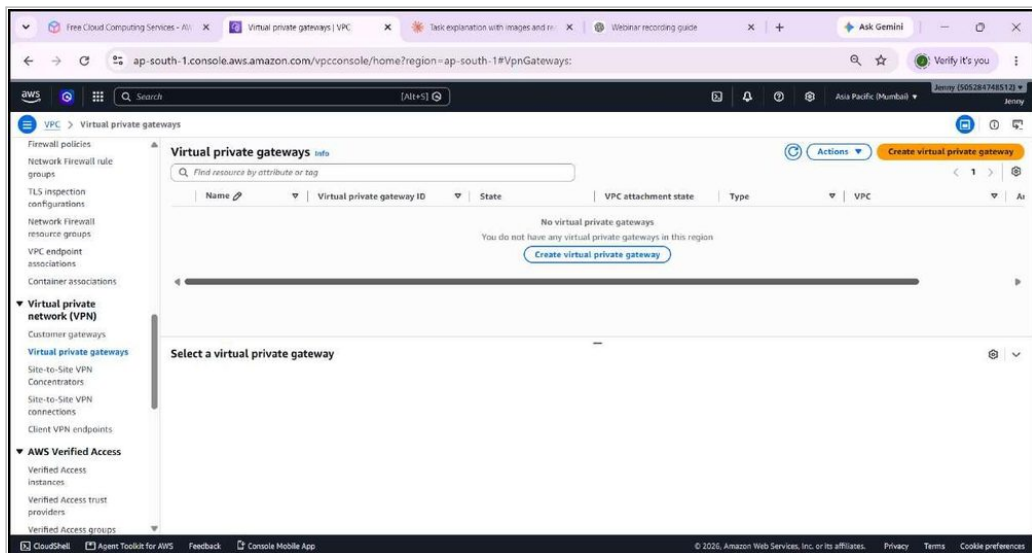
WHY The VGW, CGW and VPN connection must be in the **same region as Aws-vpc** (Mumbai).

HOW

1. Select region **Mumbai**.
2. Left menu, scroll down, click **Virtual private gateways**.



Left menu: VPN section with Virtual private gateways.



No virtual private gateways yet.

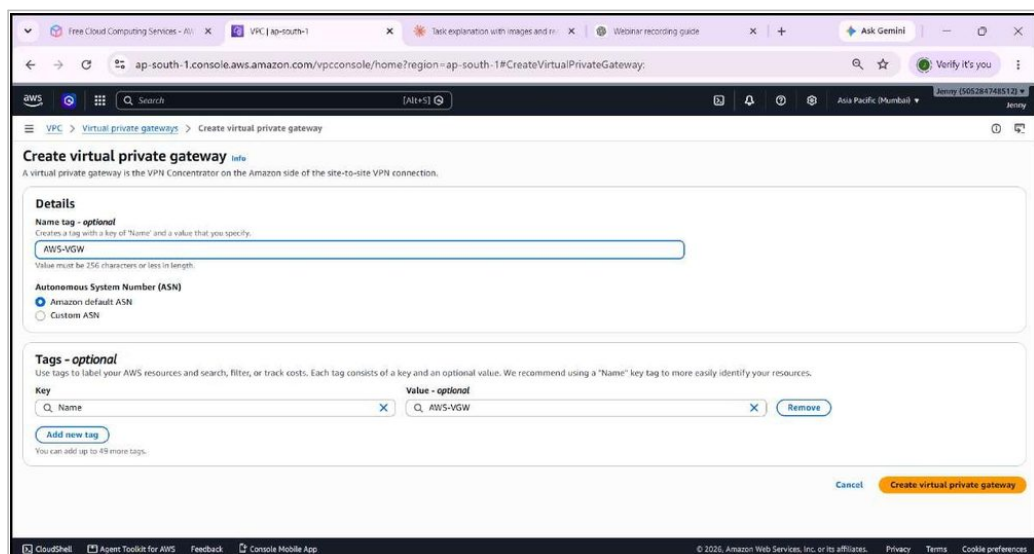
Step 17: Create the Virtual Private Gateway "AWS-VGW"

WHAT Create the AWS-side VPN endpoint.

WHY The VGW is the door of our VPC for the VPN tunnel. All VPN traffic enters and leaves AWS through it.

HOW

1. Click **Create virtual private gateway**.
2. Name tag: **AWS-VGW**.
3. ASN: **Amazon default ASN** (64512).
4. Click **Create**. State becomes Available but is **Detached**.



Create virtual private gateway AWS-VGW.

Step 18: Attach the VGW to Aws-vpc

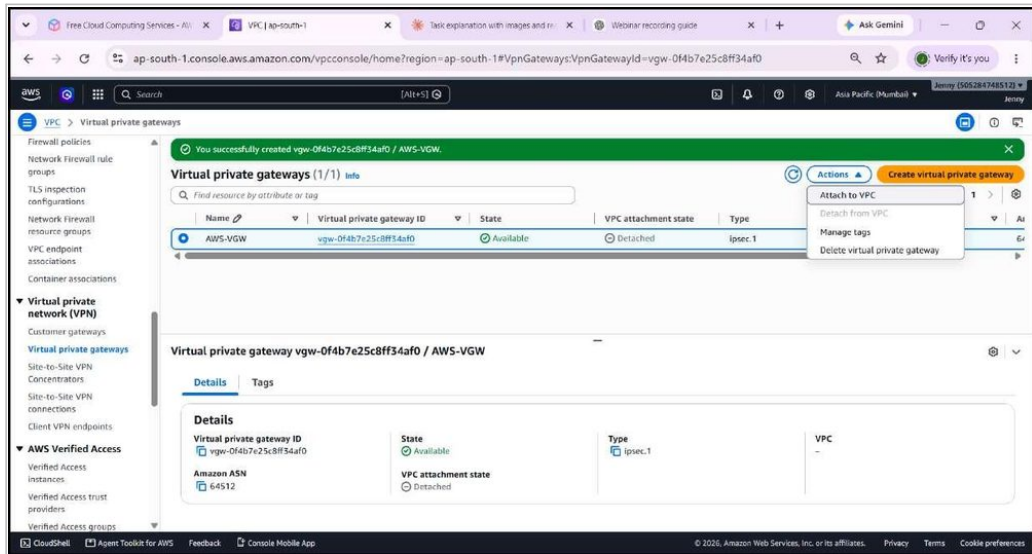
WHAT Connect AWS-VGW to the VPC.

WHY A detached VGW is connected to nothing. Attaching joins it to our network.

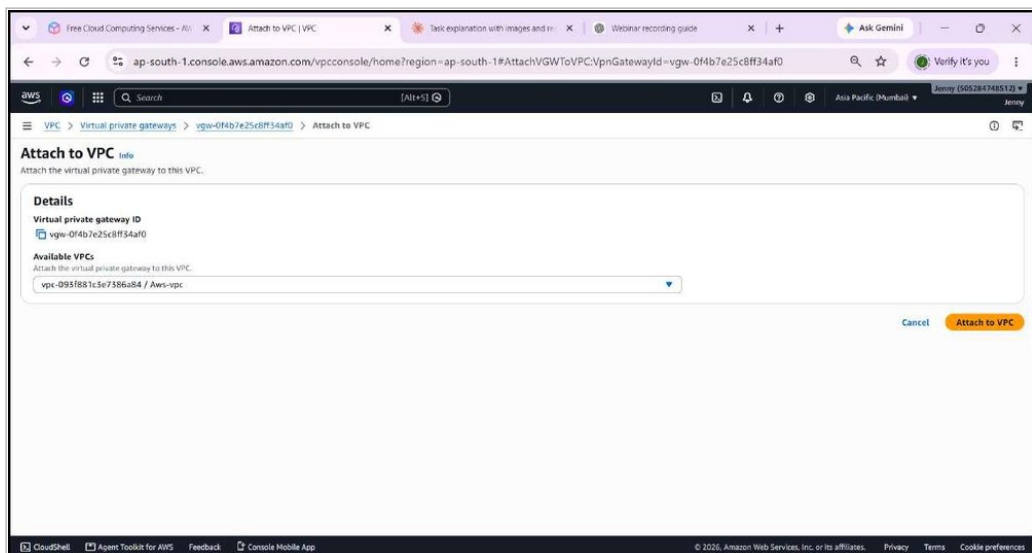
HOW

1. Select AWS-VGW, **Actions**, **Attach to VPC**.
2. Choose **Aws-vpc**.

3. Click **Attach to VPC**. Wait until the attachment shows **attached**.



VGW created. Actions, then Attach to VPC.



Select Aws-vpc.

Step 19: Add the route 192.168.0.0/16 to the VGW

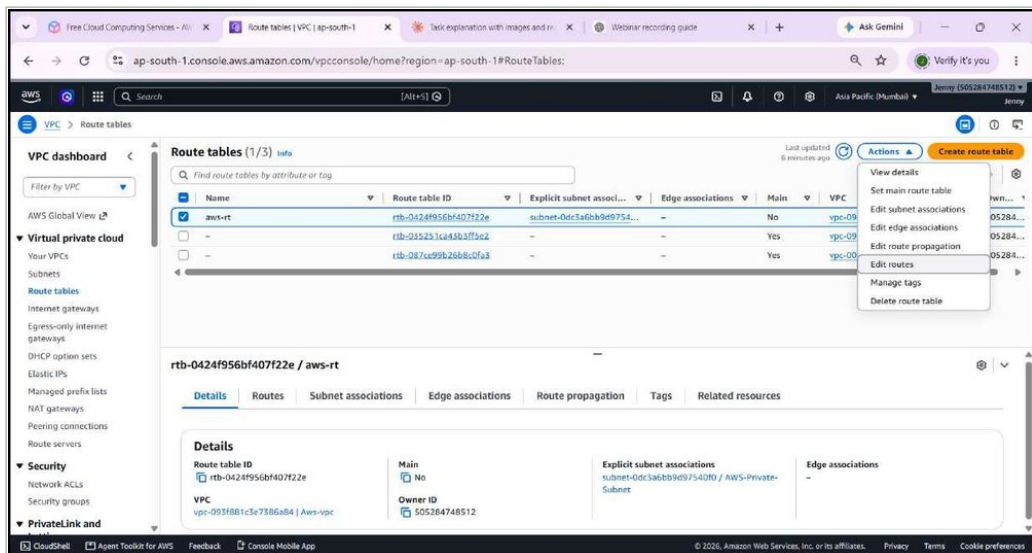
WHAT Edit route table **aws-rt** and add destination **192.168.0.0/16** with target **AWS-VGW**.

WHY Without this route, the Private EC2 does not know how to send replies to the data centre. This one line sends all data-centre traffic into the VPN.

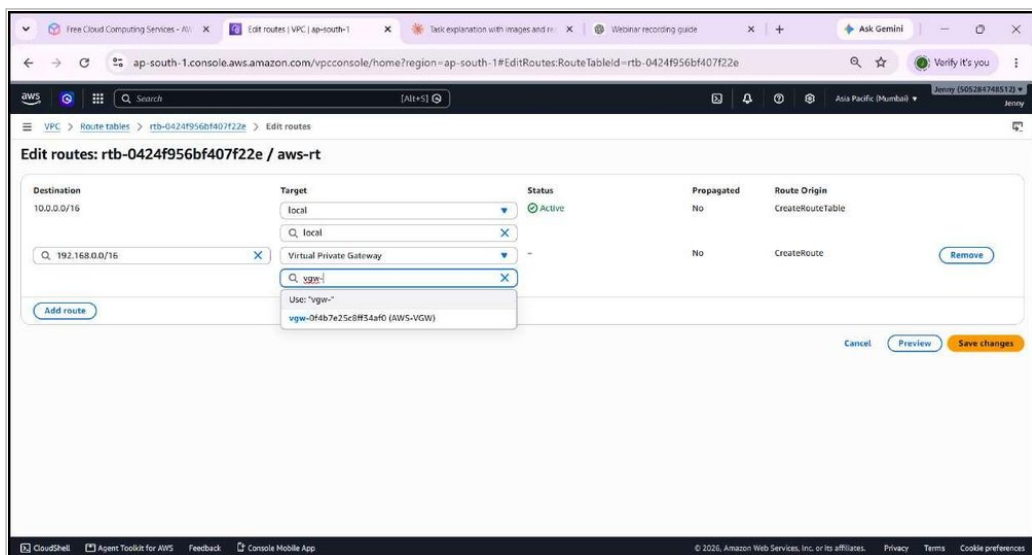
HOW

1. **Route tables**, select **aws-rt**, **Actions**, **Edit routes**.
2. **Add route**.
3. Destination: **192.168.0.0/16**. Target: **Virtual Private Gateway** then **AWS-VGW**.
4. Click **Save changes**.

Tip: Another way is **Route propagation**: AWS then adds VPN routes automatically. In this lab we added the route by hand.



aws-rt: Actions, Edit routes.



New route 192.168.0.0/16 to the VGW.

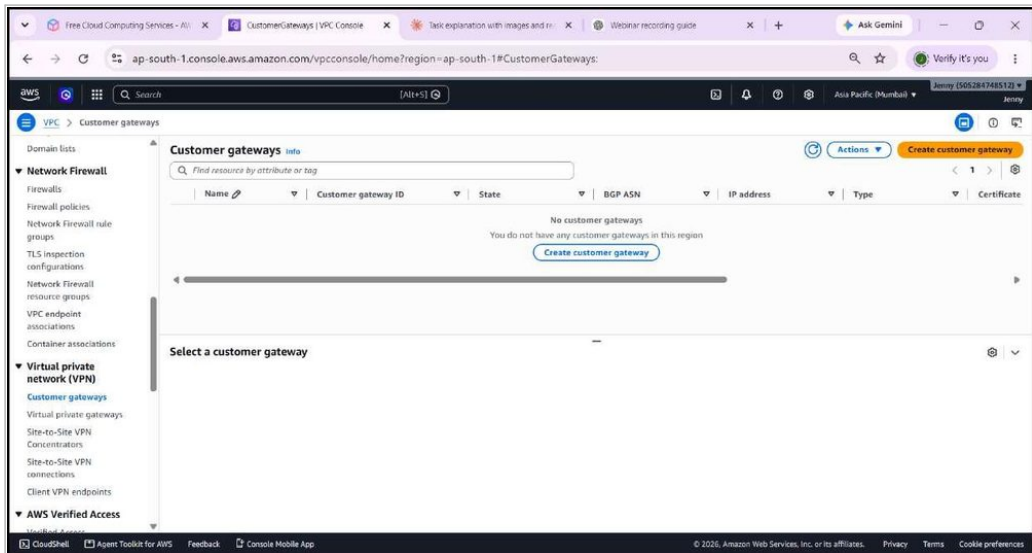
Step 20: Create the Customer Gateway "DC-CGW"

WHAT Tell AWS where our office VPN device is.

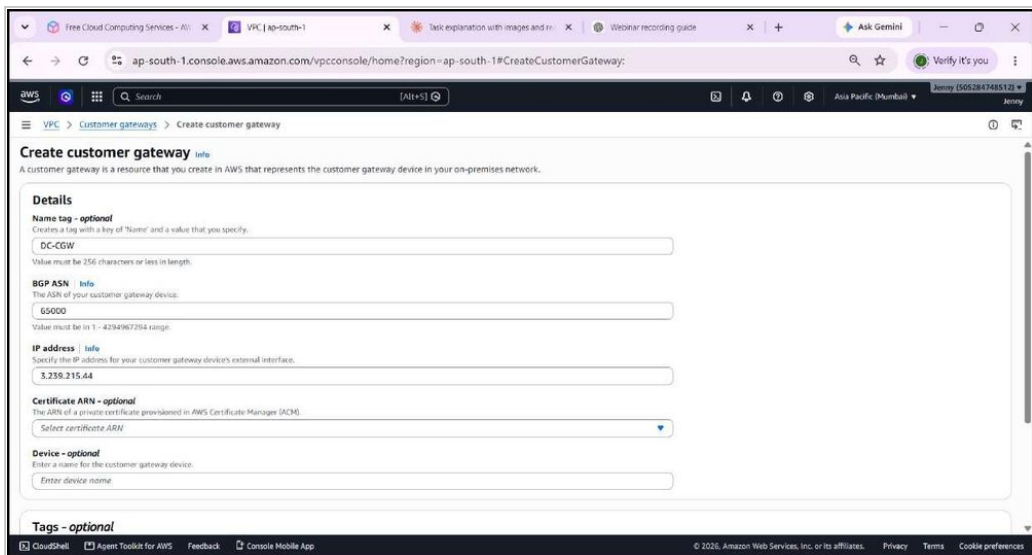
WHY AWS needs the **public IP** of our side to build the tunnel. The VPN EC2 public IP is 3.239.215.44.

HOW

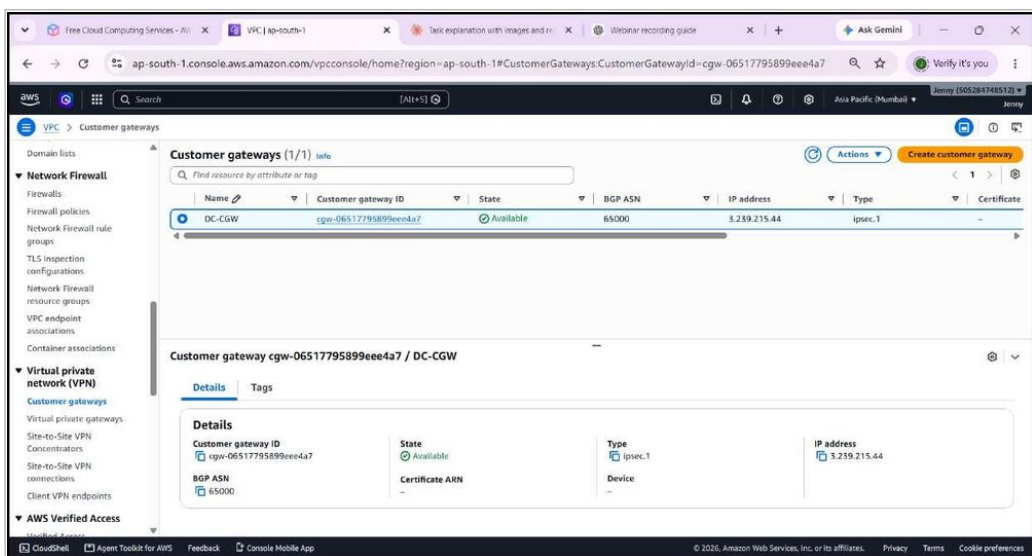
1. Left menu: **Customer gateways, Create customer gateway.**
2. Name: **DC-CGW.**
3. BGP ASN: **65000** (any private number; we use static routing so it is not really used).
4. IP address: **3.239.215.44.**
5. Leave Certificate ARN and Device empty. Click **Create.**



No customer gateway yet.



DC-CGW details: ASN 65000 and IP 3.239.215.44.



DC-CGW is Available.

Step 21: Create the Site-to-Site VPN connection

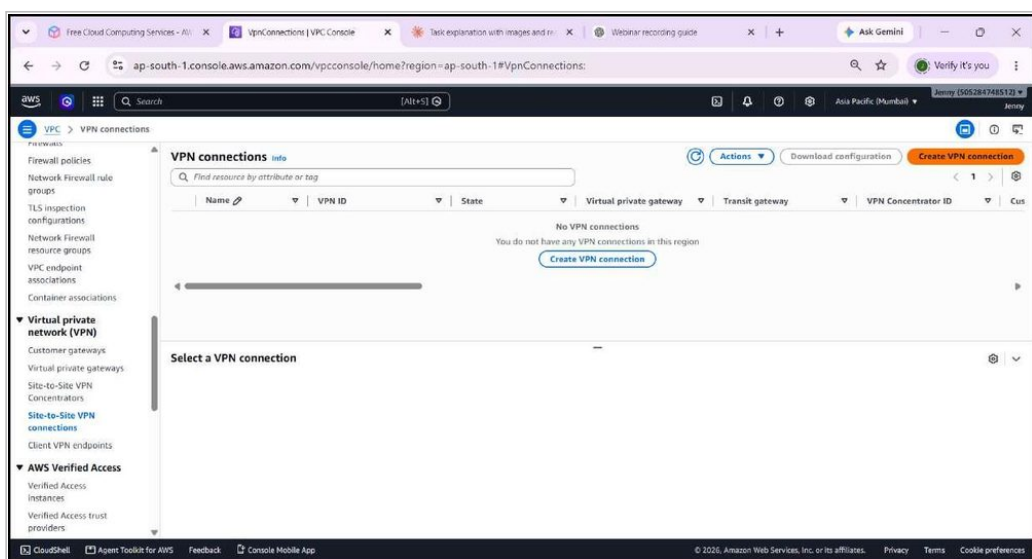
WHAT Join the VGW and the CGW with a VPN connection named **AWS-to-DC-VPN**.

WHY This creates the actual tunnels. AWS makes **two tunnels** (in two zones) so there is a backup.

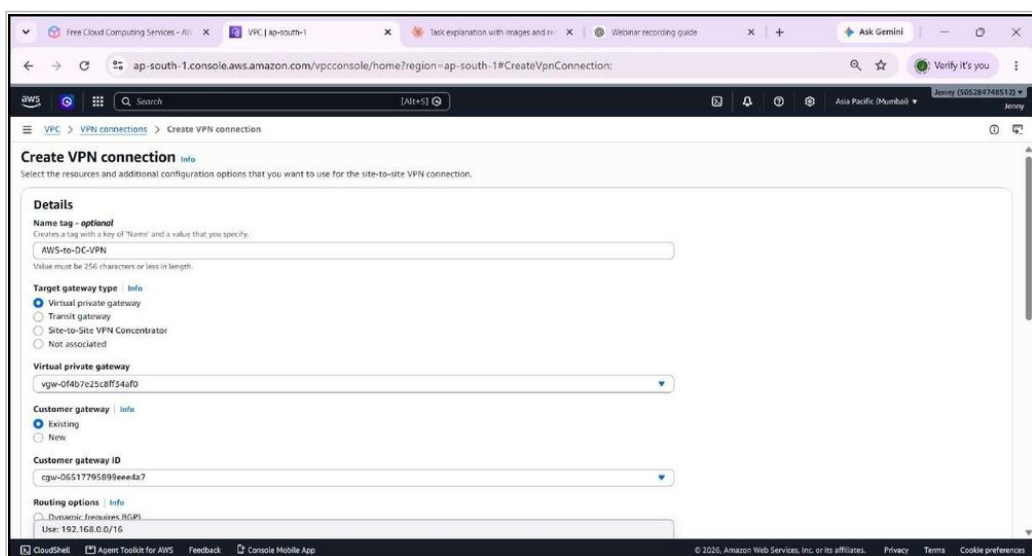
HOW

1. Left menu: **Site-to-Site VPN connections, Create VPN connection.**
2. Name: **AWS-to-DC-VPN.**
3. Target gateway type: **Virtual private gateway**, choose **AWS-VGW.**
4. Customer gateway: **Existing**, choose **DC-CGW.**
5. Routing options: **Static**. Static IP prefix: **192.168.0.0/16** (our data centre network).
6. Keep the other settings default and click **Create VPN connection**. State is **Pending** for a few minutes.

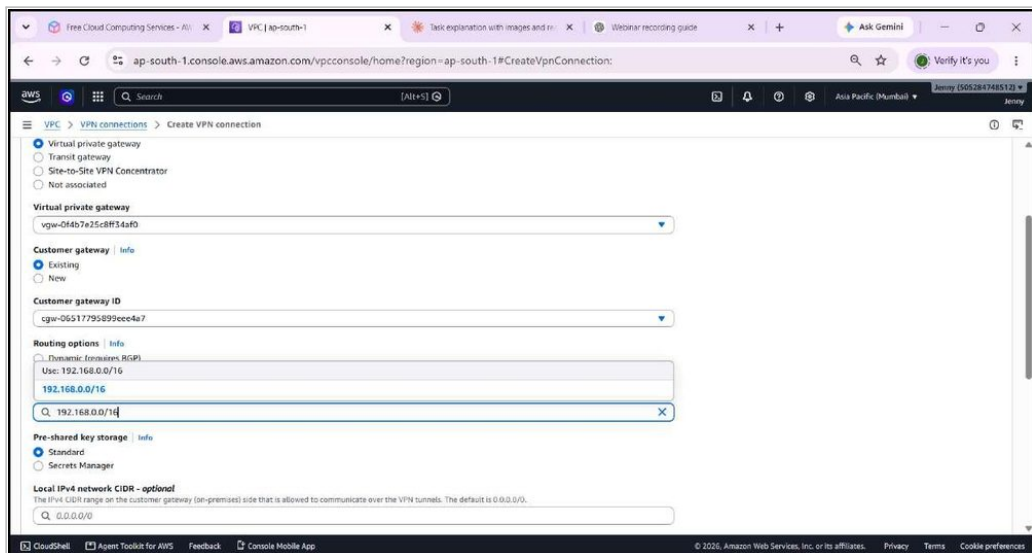
Tip: Static routing means we type the networks ourselves (simple). **Dynamic** routing uses BGP and needs extra software on the VPN server.



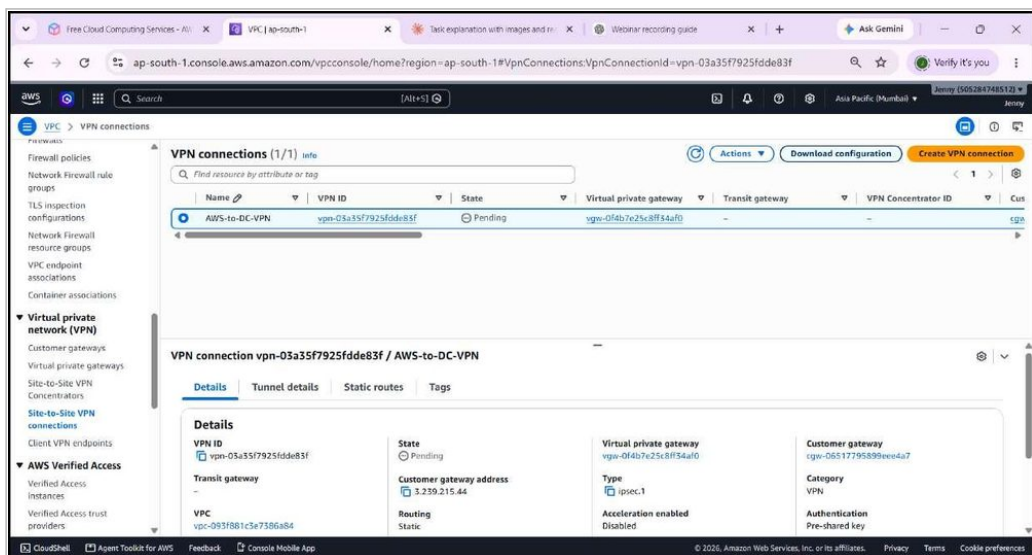
No VPN connections yet.



VPN connection: VGW, existing CGW.



Static route 192.168.0.0/16.



VPN connection created, state Pending.

Step 22: Download the configuration file

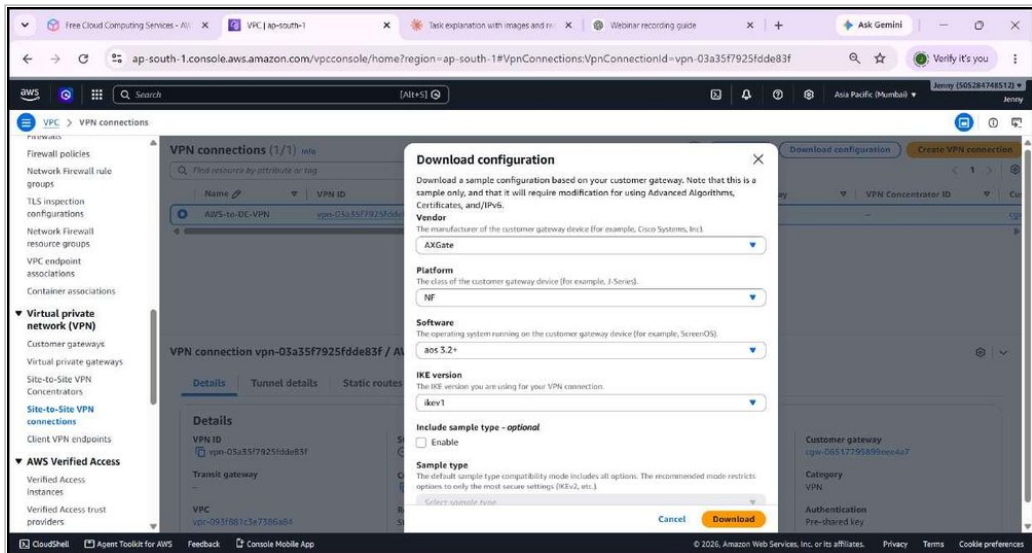
WHAT Download a ready-made config for our VPN software.

WHY AWS creates the tunnel IPs and secret keys. The file shows us exactly what to type on the VPN EC2.

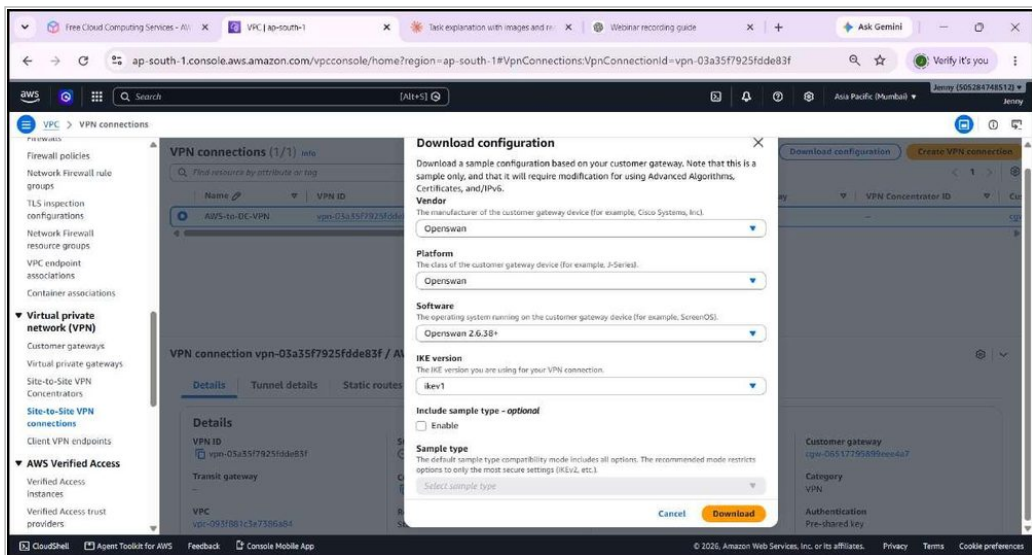
HOW

1. Select the VPN connection, click **Download configuration**.
2. Vendor **Openswan**, Platform **Openswan**, Software **Openswan 2.6.38+**, IKE version **ikev1**. (The first picture shows the default AXGate; change it.)
3. Click **Download**. A file like **vpn-03a35f7925fdde83f.txt** is saved.

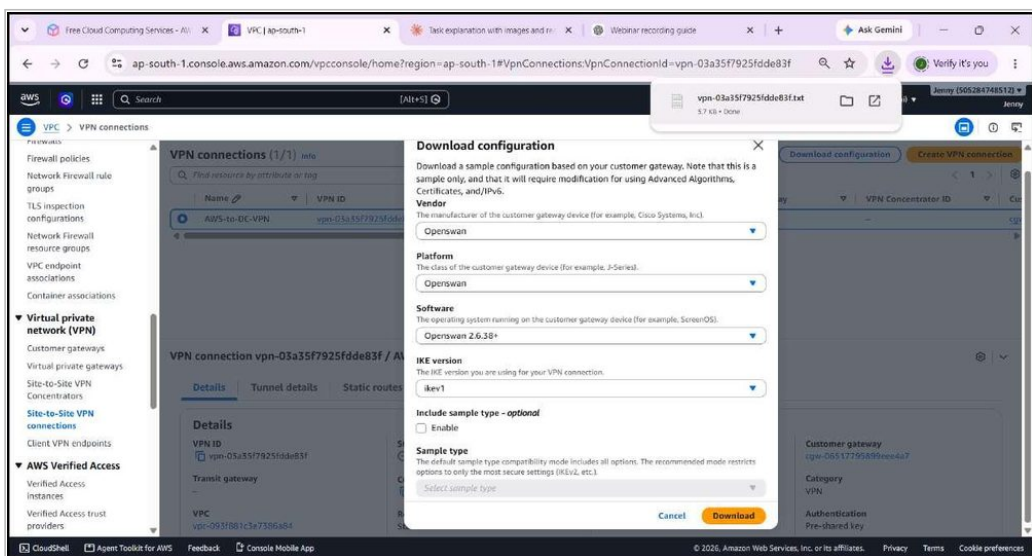
Careful: This file contains your **Pre-Shared Keys**. Treat it like a password. Do not send it to anyone.



Download configuration: the default vendor (AXGate) must be changed.



Choose Openswan for vendor, platform and software.



The .txt file is downloaded.

3. IPsec VPN Configuration (Libreswan)

Until now we only told AWS about our side. Now we must set up the VPN server (VPN EC2) so it builds the tunnel to AWS. We use **Libreswan**, a free IPsec program for Linux.

Step 23: Log in to the VPN EC2 from Windows

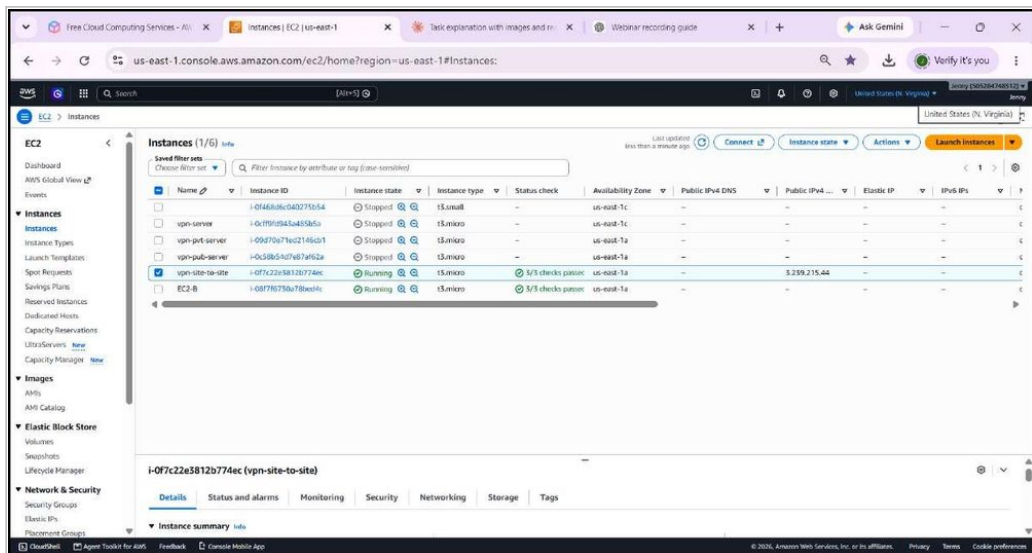
WHAT Connect to the VPN EC2 with SSH from the Windows command prompt.

WHY All VPN settings are typed inside this server.

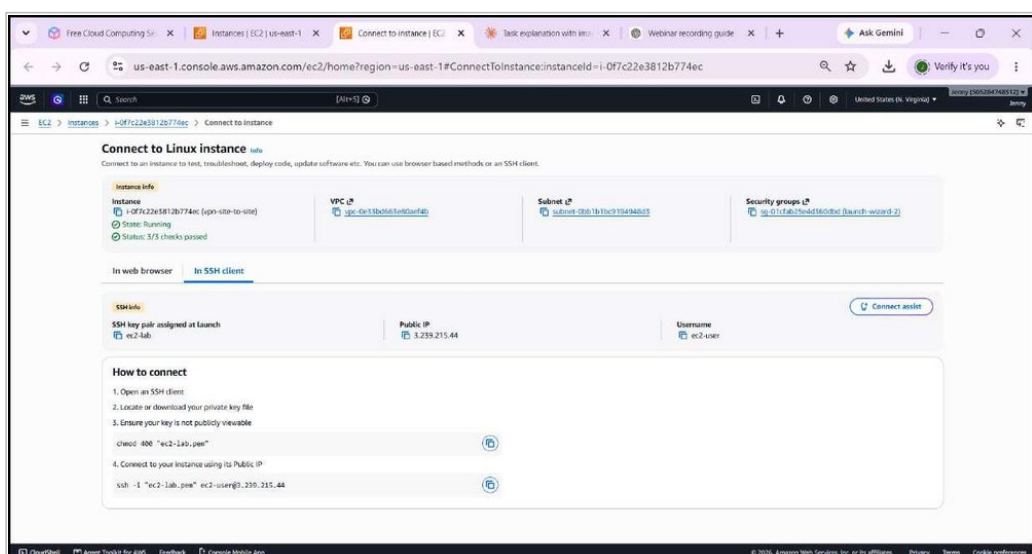
HOW

1. In EC2 (N. Virginia) select **vpn-site-to-site** and click **Connect**, tab **SSH client**. Copy the example command.
2. Open **cmd**, go to the folder where **ec2-lab.pem** is saved (Downloads).
3. Run the command below. Type **yes** when it asks about the fingerprint.
4. You see the Amazon Linux 2023 banner and the prompt **[ec2-user@ip-192-168-0-44 ~]\$**.

```
cd Downloads
ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
```



The vpn-site-to-site instance is running with public IP 3.239.215.44.



Connect page, SSH client tab.

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\URVASHI>cd Downloads
C:\Users\URVASHI\Downloads>cd ..
C:\Users\URVASHI>cd Downloads
C:\Users\URVASHI\Downloads>ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
```

Typing the SSH command in cmd.

```
C:\WINDOWS\system32\cmd.exe - ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\URVASHI>cd Downloads
C:\Users\URVASHI\Downloads>cd ..
C:\Users\URVASHI>cd Downloads
C:\Users\URVASHI\Downloads>ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
The authenticity of host '3.239.215.44 (3.239.215.44)' can't be established.
ED25519 key fingerprint is SHA256:m0A37pbJhPR5Qcvft3/J+97Y7nQuHd/ISM/GqLpTQ84.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

First time only: type yes.


```
ec2-user@ip-192-168-0-44:~$ sudo microvm console --id=ami-02680-B4572
MicroVM Windows [Version 10.0.22H2.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\USERASHI>cd Downloads
C:\Users\USERASHI>Downloads>cd ..
C:\Users\USERASHI>cd Downloads
C:\Users\USERASHI>Downloads>ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
The authenticity of host '3.239.215.44 (3.239.215.44)' can't be established.
ED25519 key fingerprint is SHA256:m0A7ph3m9d5Qcvit3/1e97YqDMM/TSH/GulpTQB4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.239.215.44' (ED25519) to the list of known hosts.
```



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-192-168-0-44 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-44 ~]$
```

581 kB/s | 82 kB 00:00

Update finished: Complete!

[illegible]

Install command.

```

ec2-user@ip-192-168-0-44:
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-44 ~]$ sudo dnf install librewan -y
Last metadata expiration check: 0:01:05 ago on Wed Sep 30 11:32:40 2026.
Dependencies resolved.

Package              Architecture      Version              Repository            Size
----
Installing:
librewan              x86_64            4.12-3.0.amzn2023.0.3  amazonlinux           1.3 M
Installing dependencies:
tdn                   x86_64            1.8.3-2.amzn2023.0.3  amazonlinux           175 k
aws-tools             x86_64            3.112.0-8.amzn2023.0.2  amazonlinux           437 k
unbound-libs          x86_64            1.17.1-1.amzn2023.0.15  amazonlinux           338 k
Installing weak dependencies:
unbound-anchor        x86_64            1.17.1-1.amzn2023.0.15  amazonlinux           38 k

Transaction Summary
-----
Install 5 Packages

Total download size: 2.5 M
Installed size: 6.2 M
Downloading Packages:
(1/5): lds-1.8.3-2.amzn2023.0.3.x86_64.rpm                5.6 MB/s | 175 kB  00:00
(2/5): aws-tools-3.112.0-8.amzn2023.0.2.x86_64.rpm        11 MB/s | 437 kB  00:00
(3/5): unbound-libs-1.17.1-1.amzn2023.0.15.x86_64.rpm      16 MB/s | 338 kB  00:00
(4/5): librewan-4.12-3.amzn2023.0.3.x86_64.rpm             45 MB/s | 1.3 MB  00:00
(5/5): unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64.rpm    529 kB/s | 38 kB   00:00
-----
Total                                                                16 MB/s | 2.5 MB  00:00

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing...                                                        1/1
Running scriptlet: unbound-libs-1.17.1-1.amzn2023.0.15.x86_64    1/5
Installing...                                                       2/5
Installing...                                                       3/5
Running scriptlet: unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Running scriptlet: unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Created symlink /etc/systemd/system/timers.target.wants/unbound-anchor.timer + /usr/lib/systemd/system/unbound-anchor.timer.

Installing...                                                       3/5
Installing...                                                       4/5
Installing...                                                       5/5
Running scriptlet: librewan-4.12-3.amzn2023.0.3.x86_64          3/5
Running scriptlet: librewan-4.12-3.amzn2023.0.3.x86_64          4/5
Running scriptlet: librewan-4.12-3.amzn2023.0.3.x86_64          5/5

```

Packages being installed.

```
ec2-user@ip-192-168-0-44:~$ sudo yum install unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64.rpm
Total download size: 2.5 M
Installed size: 8.2 M
Downloading Packages:
(1/5): idns-1.8.3-2.amzn2023.0.3.x86_64.rpm 5.6 MB/s | 175 kB 00:00
(2/5): nss-tools-3.112.0-8.amzn2023.0.2.x86_64.rpm 11 MB/s | 437 kB 00:00
(3/5): unbound-libs-1.17.1-1.amzn2023.0.15.x86_64.rpm 16 MB/s | 538 kB 00:00
(4/5): libreswan-4.12-3.amzn2023.0.3.x86_64.rpm 15 MB/s | 1.3 MB 00:00
(5/5): unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64.rpm 529 kB/s | 38 kB 00:00
Total 36 MB/s | 2.5 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Running scriptlet: unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 1/5
Installing : unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 1/5
Installing : unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Running scriptlet: unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Created symlink /etc/systemd/system/timers.target.wants/unbound-anchor.timer → /usr/lib/systemd/system/unbound-anchor.timer.
Installing : nss-tools-3.112.0-8.amzn2023.0.2.x86_64 3/5
Installing : idns-1.8.3-2.amzn2023.0.3.x86_64 4/5
Installing : libreswan-4.12-3.amzn2023.0.3.x86_64 5/5
Running scriptlet: libreswan-4.12-3.amzn2023.0.3.x86_64 5/5
Verifying : idns-1.8.3-2.amzn2023.0.3.x86_64 1/5
Verifying : libreswan-4.12-3.amzn2023.0.3.x86_64 2/5
Verifying : nss-tools-3.112.0-8.amzn2023.0.2.x86_64 3/5
Verifying : unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 4/5
Verifying : unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 5/5
Installed:
idns-1.8.3-2.amzn2023.0.3.x86_64  libreswan-4.12-3.amzn2023.0.3.x86_64  nss-tools-3.112.0-8.amzn2023.0.2.x86_64  unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64  unbound-libs-1.17.1-1.amzn2023.0.15.x86_64
Complete!
ec2-user@ip-192-168-0-44:~$
```

Install finished.

```
Select ec2-user@ip-192-168-0-44:~$ sudo yum install unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64.rpm
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Running scriptlet: unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 1/5
Installing : unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 1/5
Installing : unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Running scriptlet: unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 2/5
Created symlink /etc/systemd/system/timers.target.wants/unbound-anchor.timer → /usr/lib/systemd/system/unbound-anchor.timer.
Installing : nss-tools-3.112.0-8.amzn2023.0.2.x86_64 3/5
Installing : idns-1.8.3-2.amzn2023.0.3.x86_64 4/5
Installing : libreswan-4.12-3.amzn2023.0.3.x86_64 5/5
Running scriptlet: libreswan-4.12-3.amzn2023.0.3.x86_64 5/5
Verifying : idns-1.8.3-2.amzn2023.0.3.x86_64 1/5
Verifying : libreswan-4.12-3.amzn2023.0.3.x86_64 2/5
Verifying : nss-tools-3.112.0-8.amzn2023.0.2.x86_64 3/5
Verifying : unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64 4/5
Verifying : unbound-libs-1.17.1-1.amzn2023.0.15.x86_64 5/5
Installed:
idns-1.8.3-2.amzn2023.0.3.x86_64  libreswan-4.12-3.amzn2023.0.3.x86_64  nss-tools-3.112.0-8.amzn2023.0.2.x86_64  unbound-anchor-1.17.1-1.amzn2023.0.15.x86_64  unbound-libs-1.17.1-1.amzn2023.0.15.x86_64
Complete!
ec2-user@ip-192-168-0-44:~$ ipsec --version
libreswan 4.12
ec2-user@ip-192-168-0-44:~$
```

Version is Libreswan 4.12.

Step 25: Read the downloaded configuration file

WHAT Open the downloaded .txt file in Notepad and understand it.

WHY It has two tunnels. Each tunnel has 5 small steps. We must copy the right values from it.

HOW

1. Open the file. At the top you see the VPN ID, VGW ID and CGW ID.
2. Find **IPSEC Tunnel #1** and **IPSEC Tunnel #2**.
3. In each tunnel copy: **leftid** (our public IP), **right** (AWS tunnel IP) and the **PSK** line.

```
vpn-03a35f7925fddde83f.txt
File Edit View
H1 B I S
Amazon Web Services
Virtual Private Cloud

AWS utilizes unique identifiers to manipulate the configuration of
a VPN Connection. Each VPN Connection is assigned an identifier and is
associated with two other identifiers, namely the
Customer Gateway Identifier and Virtual Private Gateway Identifier.

Your VPN Connection ID      : vpn-03a35f7925fddde83f
Your Virtual Private Gateway ID : vgw-0f4b7e25c8ff34af0
Your Customer Gateway ID    : cgw-06517795899eee4a7

This configuration consists of two tunnels. Both tunnels must be
configured on your Customer Gateway, but only one of those tunnels should be up at any given time.

At this time this configuration has only been tested for Openswan 2.6.38 or later, but may work with earlier versions.

-----
IPSEC Tunnel #1
-----

This configuration assumes that you already have a default openswan installation in place on the Amazon Linux operating system (but
may also work with other distros as well)

1) Open /etc/sysctl.conf and ensure that its values match the following:
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0

Ln 61, Col 42  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Top of the file: the IDs.

```
vpn-03a35f7925fddde83f.txt
File Edit View
H1 B I S
IPSEC Tunnel #1
-----

This configuration assumes that you already have a default openswan installation in place on the Amazon Linux operating system (but
may also work with other distros as well)

1) Open /etc/sysctl.conf and ensure that its values match the following:
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0

2) Apply the changes in step 1 by executing the command 'sysctl -p'

3) Open /etc/ipsec.conf and look for the line below. Ensure that the # in front of the line has been removed, then save and exit the
file.
#include /etc/ipsec.d/*.conf

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration to the
end in the file:
#leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with this value
(don't include the brackets). If you have multiple subnets, you can use 0.0.0.0/0 instead.
#rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you will need
to replace <REMOTE NETWORK> with this value (don't include brackets).

conn Tunnel1
    authby=secret
    auto=start
    left=%defaultroute
    leftid=3.239.215.44
    right=13.201.140.220

Ln 61, Col 42  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Tunnel 1: steps 1 to 4.

```
vpn-03a35f7925fddde83f.txt
File Edit View
H1 B I S
t110.
#include /etc/ipsec.d/*.conf

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration to the
end in the file:
#leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with this value
(don't include the brackets). If you have multiple subnets, you can use 0.0.0.0/0 instead.
#rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you will need
to replace <REMOTE NETWORK> with this value (don't include brackets).

conn Tunnel1
    authby=secret
    auto=start
    left=%defaultroute
    leftid=3.239.215.44
    right=13.201.140.220
    type=tunnel
    ikelifetime=8h
    keylifetime=1h
    phase2alg=aes128-sha1;modp1024
    ike=aes128-sha1;modp1024
    auth=esp
    keyingtries=%forever
    keyexchange=ike
    leftsubnet=<LOCAL NETWORK>
    rightsubnet=<REMOTE NETWORK>
    dpddelay=10
    dpdtimeout=30
    dpdaction=restart_by_peer

5) Create a new file at /etc/ipsec.d/aws.secrets if it doesn't already exist, and append this line to the file (be mindful of the

Ln 61, Col 42  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Tunnel 1: the conn block starts.

```
-----
IPSEC Tunnel #2
-----

This configuration assumes that you already have a default openswan installation in place on the Amazon Linux operating system (but may also work with other distros as well)

1) Open /etc/sysctl.conf and ensure that its values match the following:
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0

2) Apply the changes in step 1 by executing the command 'sysctl -p'

3) Open /etc/ipsec.conf and look for the line below. Ensure that the # in front of the line has been removed, then save and exit the file.
    #include /etc/ipsec.d/*.conf

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration to the end in the file:
#leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with this value (don't include the brackets). If you have multiple subnets, you can use 0.0.0.0/0 instead.
#rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you will need to replace <REMOTE NETWORK> with this value (don't include brackets).

conn Tunnel2
    authby=secret
    auto=start
    left=%defaultroute
    leftid=3.239.215.44
    right=65.0.172.148

Ln 61, Col 42  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Tunnel 1 full conn block.

```
-----
IPSEC Tunnel #2
-----

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration to the end in the file:
#leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with this value (don't include the brackets). If you have multiple subnets, you can use 0.0.0.0/0 instead.
#rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you will need to replace <REMOTE NETWORK> with this value (don't include brackets).

conn Tunnel2
    authby=secret
    auto=start
    left=%defaultroute
    leftid=3.239.215.44
    right=65.0.172.148
    type=tunnel
    ikelifetime=8h
    keylife=1h
    phase2alg=aes128-sha1;modp1024
    ike=aes128-sha1;modp1024
    auth=esp
    keyingtries=%forever
    keyexchange=ike
    leftsubnet=<LOCAL NETWORK>
    rightsubnet=<REMOTE NETWORK>
    dpddelay=10
    dpdtimeout=30
    dpdaction=restart_by_peer

5) Create a new file at /etc/ipsec.d/aws.secrets if it doesn't already exist, and append this line to the file (be mindful of the spacing!):

Ln 61, Col 42  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Tunnel 2 steps.

```
-----
(OPTIONAL CONFIG) Tunnel Healthcheck and Failover
-----

Openswan does not provide a built-in tunnel failover functionality. However, there are some third-party workarounds to this.

=== DISCLAIMER ===
Please be aware that AWS is in no way responsible for any of the use, management, maintenance, or potential issues you may encounter with the third-party workarounds. It is strongly recommended that you thoroughly test any failover solution prior to implementing it into your production environment

Additional Notes and Questions
- Amazon Virtual Private Cloud Getting Started Guide:
  http://docs.amazonwebservices.com/AmazonVPC/latest/GettingStartedGuide
- Amazon Virtual Private Cloud Network Administrator Guide:
  http://docs.amazonwebservices.com/AmazonVPC/latest/NetworkAdminGuide

Ln 90, Col 12  5,839 characters  Plain text  100%  Unix (LF)  UTF-8
```

Tunnel 2 full conn block.

What the important lines mean

Line	Easy meaning
authby=secret	Prove identity with a Pre-Shared Key (PSK).
auto=start	Start the tunnel automatically when the service starts.
left=%defaultroute	Our side: use the server's default network interface.
leftid=3.239.215.44	Our public IP, used as our identity.
right=13.201.140.220	AWS tunnel endpoint IP (Tunnel 2 uses 65.0.172.148).
type=tunnel	Full tunnel mode: whole packets are wrapped and encrypted.
ikelifetime=8h / keylife=1h	How long keys live before they are renewed (security).
ike=... / phase2alg=...	Encryption and hash methods for Phase 1 and Phase 2 (aes128, sha1).
keyingtries=%forever	Keep trying to connect forever if it fails.
leftsubnet / rightsubnet	Which networks may use the tunnel: our network and the remote network.
dpddelay / dpdtimeout / dpdaction	Dead Peer Detection: check every 10 s if the other end is alive; restart if dead.

The file has two placeholders: <LOCAL NETWORK> and <REMOTE NETWORK>. We replace them with our own networks:

Placeholder	Our value	Meaning
<LOCAL NETWORK>	192.168.0.0/16	The data centre network behind the VPN EC2
<REMOTE NETWORK>	10.0.0.0/16	The AWS network (Aws-vpc) we want to reach

Step 26: Apply the configuration on the VPN EC2

WHAT Edit 4 places: kernel settings, ipsec.conf, aws.conf and aws.secrets. Then start the service.

WHY This tells Libreswan which tunnel to build, with which keys, and lets the server pass packets between networks.

HOW

1. **Kernel settings:** open `/etc/sysctl.conf`, add the 3 lines below, then run **sudo sysctl -p**.
2. **ipsec.conf:** open `/etc/ipsec.conf` and make sure the line **include /etc/ipsec.d/*.conf** has no `#` in front.
3. **aws.conf:** create `/etc/ipsec.d/aws.conf` with the Tunnel1 block below (our two networks filled in).
4. **aws.secrets:** create `/etc/ipsec.d/aws.secrets` with the PSK line from the downloaded file and protect it with `chmod 600`.
5. Enable and start the service, then check the status.

1) Kernel settings

```
sudo nano /etc/sysctl.conf
# add these lines at the end
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0

sudo sysctl -p
```

2) ipsec.conf

```
sudo nano /etc/ipsec.conf
# make sure this line is active (no # in front)
include /etc/ipsec.d/*.conf
```

3) aws.conf (Tunnel 1)

```
sudo nano /etc/ipsec.d/aws.conf

conn Tunnel1
    authby=secret
    auto=start
    left=%defaultroute
    leftid=3.239.215.44
    right=13.201.140.220
    type=tunnel
    ikelifetime=8h
    keylife=1h
    phase2alg=aes128-sha1;modp1024
    ike=aes128-sha1;modp1024
    auth=esp
    keyingtries=%forever
    keyexchange=ike
    leftsubnet=192.168.0.0/16
    rightsubnet=10.0.0.0/16
    dpddelay=10
    dpdtimeout=30
    dpdaction=restart_by_peer
```

4) aws.secrets (copy the real PSK from your downloaded file)

```
sudo nano /etc/ipsec.d/aws.secrets

3.239.215.44 13.201.140.220: PSK "<YOUR-PSK-FOR-TUNNEL-1>"

sudo chmod 600 /etc/ipsec.d/aws.secrets
```

5) Start and check

```
sudo systemctl enable --now ipsec
sudo systemctl status ipsec
sudo ipsec status
```

Why those 3 kernel lines? `ip_forward=1`: the server may pass packets between networks (router mode). `rp_filter=0`: do not drop packets that arrive on an unexpected path. `accept_source_route=0`: ignore source-routed packets (a safety setting).

Important: AWS says: both tunnels are in the file, but **only one should be up at a time** on Openswan/Libreswan. So we use **Tunnel1**. Tunnel2 is the backup: you may add the same block named **conn Tunnel2** (with `right=65.0.172.148` and its own PSK line) but set **auto=add** so it is loaded but not started.

Step 27: Check that the tunnel is UP

WHAT Check the tunnel in Linux and in the AWS console.

WHY We must be sure the tunnel is built before we test ping.

HOW

1. On the VPN EC2 run **sudo ipsec status**. Look for the words **IPsec SA established**.
2. Run **sudo ipsec trafficstatus** to see bytes in and out.
3. Run **sudo ip xfrm state** and **sudo ip xfrm policy** to see the kernel part (XFRM).
4. In AWS (Mumbai) open the VPN connection, tab **Tunnel details**. Tunnel 1 should be **Up**.

```
sudo ipsec status
sudo ipsec trafficstatus
sudo ip xfrm state
sudo ip xfrm policy
```

Example output (your numbers and names may look a little different)

```
"Tunnel1": 192.168.0.0/16===192.168.0.44[3.239.215.44]...13.201.140.220===10.0.0.0/16; erouted
#2: "Tunnel1":4500 STATE_V1_QUICK_I2 (IPsec SA established); EVENT_SA_REPLACE in 3300s
#1: "Tunnel1":4500 STATE_V1_MAIN_I4 (ISAKMP SA established); EVENT_SA_REPLACE in 27800s

$ sudo ip xfrm policy
src 192.168.0.0/16 dst 10.0.0.0/16
  dir out priority 1042 ptype main
  tmpl src 192.168.0.44 dst 13.201.140.220
    proto esp reqid 16389 mode tunnel
```

What is XFRM?

XFRM is the part of the Linux **kernel** that really encrypts and decrypts packets. Libreswan (a program called *pluto*) only does the talking: it agrees the keys with AWS. Then it hands the keys to XFRM. After that, the kernel does the fast work for every packet.

XFRM part	Command	What it shows
State (SA)	ip xfrm state	Who the tunnel ends are, the SPI number, the encryption method and the keys in use.
Policy	ip xfrm policy	The rule: "traffic from 192.168.0.0/16 to 10.0.0.0/16 must go through the tunnel".

How the tunnel was established (simple story)

1. **VPN EC2 starts Libreswan.** Because of `auto=start`, it contacts AWS tunnel IP 13.201.140.220.
2. **Phase 1 (IKE):** both sides prove who they are with the PSK, agree on encryption and make a safe control channel (ISAKMP SA).
3. **Phase 2 (IPsec):** inside that channel they agree on keys for real data (IPsec SA, using ESP).
4. **Libreswan gives the keys to the kernel (XFRM)** with the policy for 192.168.0.0/16 and 10.0.0.0/16.
5. **AWS shows the tunnel as UP.** Now any packet between the two networks is wrapped in ESP and encrypted.
6. **DPD** keeps checking the tunnel every 10 seconds; if it dies, it restarts.

4. Routing

A VPN tunnel alone is not enough. Every machine must also **know the way** to the other network. This is what routing does.

4.1 AWS route table (aws-rt)

Destination	Target	Why this route is required
10.0.0.0/16	local	Created automatically. Servers inside Aws-vpc talk to each other.
192.168.0.0/16	AWS-VGW	Any packet for the data centre must enter the VPN. Without it the Private EC2 can receive requests but its replies get lost .

4.2 What happens at the Private EC2 (10.0.0.164)

The Private EC2 gets a ping from 192.168.0.44. It must answer to 192.168.0.44. It checks its subnet route table (aws-rt), sees the rule 192.168.0.0/16 to AWS-VGW, and sends the reply to the VGW. The VGW puts it in the tunnel. No extra setting is needed on the Private EC2 itself.

4.3 Data centre side

Place	Rule	Why
DC route tables	192.168.0.0/16 local, 0.0.0.0/0 to DC-IGW (public one)	The VPN EC2 needs the internet to reach the AWS tunnel IP 13.201.140.220.
VPN EC2 (Libreswan)	XFRM policy: 192.168.0.0/16 to 10.0.0.0/16 through the tunnel	This is the "route" for VPN traffic. It lives in the kernel, not in the AWS route table.

Tip: Extra: If EC2-B (or any other office computer) must also reach 10.0.0.164, add this route in DC-PVT-RT: 10.0.0.0/16 to the **VPN EC2 network interface**, and turn **Source/destination check = Stop** on the VPN EC2. The VPN EC2 then works as the office gateway. In our main tests the VPN EC2 itself sends the traffic, so this was not needed.

4.4 Tunnel vs normal routing

Normal routing	Tunnel routing
Packet goes as plain text through many routers.	Packet is encrypted and put inside a new packet (ESP).
Private IPs (10.x, 192.168.x) cannot travel on the internet.	Private IPs are hidden inside the tunnel, so they work across the internet.
Anyone on the path can read it.	Only the two tunnel ends can read it.

5. Testing & Troubleshooting

We test step by step: first the tunnel, then ping, then fix problems one by one. Run all tests from the **VPN EC2** (the office side).

5.1 Test 1: Is the tunnel up?

```
sudo ipsec status | grep -i established
sudo ipsec trafficstatus
```

You must see **IPsec SA established**. Also check AWS console, VPN connection, **Tunnel details**: Tunnel 1 = Up.

5.2 Test 2: First ping fails (wrong IP)

```
ping -c 4 10.0.0.1
```

Example output (your numbers and names may look a little different)

```
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3070ms
```

Reason: 10.0.0.1 is **not a server**. In every AWS subnet the first addresses are reserved: .0 network, .1 VPC router, .2 DNS, .3 future use. The VPC router does not answer ping from outside. Our real Private EC2 is **10.0.0.164** (see EC2 console, Private IPv4 address).

5.3 Test 3: Correct IP but still failing (Security Group)

```
ping -c 4 10.0.0.164
```

Example output (your numbers and names may look a little different)

```
PING 10.0.0.164 (10.0.0.164) 56(84) bytes of data.

--- 10.0.0.164 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss
```

Reason: The Private EC2 security group had only SSH (22). Ping uses **ICMP**, which was **not allowed**. The tunnel was fine, but the firewall of the server dropped the ping.

1. EC2 console (Mumbai), select **EC2-A**, tab **Security**, click the security group.
2. **Edit inbound rules, Add rule:** Type **All ICMP - IPv4**, Source **192.168.0.0/16**.
3. Save rules and ping again.

```
ping -c 4 10.0.0.164
```

Example output (your numbers and names may look a little different)

```
64 bytes from 10.0.0.164: icmp_seq=1 ttl=127 time=240 ms
64 bytes from 10.0.0.164: icmp_seq=2 ttl=127 time=238 ms
64 bytes from 10.0.0.164: icmp_seq=3 ttl=127 time=239 ms
64 bytes from 10.0.0.164: icmp_seq=4 ttl=127 time=238 ms

4 packets transmitted, 4 received, 0% packet loss
```

Final result: ping works. The time is high (about 240 ms) because the packets travel from N. Virginia to Mumbai.

5.4 Deeper checks (if something still fails)

```
# Firewall on the VPN EC2
sudo iptables -L -n -v
systemctl status firewalld

# See the ping packets leaving
sudo tcpdump -ni any icmp

# See the encrypted packets (ESP / NAT-T)
sudo tcpdump -ni any "esp or udp port 4500 or udp port 500"

# Kernel VPN state and counters
sudo ip -s xfrm state
sudo ip xfrm policy
```

Check	Good result	Bad result means
iptables -L -n -v	Policy ACCEPT, no DROP rules for our traffic	A firewall rule blocks ICMP or ESP
firewalld status	Not installed / inactive (default on Amazon Linux 2023)	If active, allow ipsec service
tcpdump icmp	echo request and echo reply both seen	Only requests: reply is lost (check SG and aws-rt route)
tcpdump esp / 4500	ESP or UDP 4500 packets flow while you ping	Nothing: traffic is not entering the tunnel (check policy and ip_forward)
ip -s xfrm state	Byte and packet counters keep growing	Counters stay 0: tunnel not used

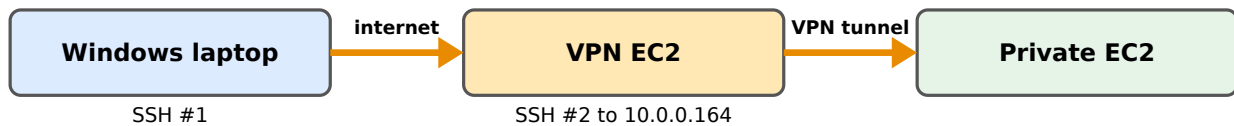
5.5 Mistake, reason, solution, final result

Mistake / problem	Reason	Solution	Final result
Ping 10.0.0.1 failed	.1 is the VPC router, not our server	Use the real Private IP 10.0.0.164	Correct target address
Ping 10.0.0.164 still failed	Security group had no ICMP rule	Add All ICMP-IPv4 from 192.168.0.0/16	Ping works, 0% loss
Other things to check: tunnel Down	PSK or IP typed wrong, or UDP 500/4500 blocked, or public IP changed	Fix aws.secrets / aws.conf, open UDP 500 and 4500, update CGW IP	Tunnel 1 = Up
Other: ping goes nowhere	Missing route 192.168.0.0/16 to VGW in aws-rt	Add the route	Replies return
Other: no traffic in tunnel	sysctl settings not applied	Run sudo sysctl -p	ESP packets seen

"Other" rows are common problems to check; the first two rows are what happened in this lab.

6. SSH Connectivity

We use SSH two times: first from Windows to the VPN EC2 (over the internet), then from the VPN EC2 to the Private EC2 (through the VPN tunnel).



6.1 Windows to VPN EC2

```
cd Downloads
ssh -i "ec2-lab.pem" ec2-user@3.239.215.44
```

Prompt after login: [ec2-user@ip-192-168-0-44 ~]\$. The VPN EC2 has a **public IP**, so this works from anywhere.

6.2 VPN EC2 to Private EC2

The Private EC2 has **no public IP**. You can only reach it from the VPN EC2 because the tunnel carries the traffic to 10.0.0.164.

Key handling: the two servers use **different key pairs**. The VPN EC2 uses `ec2-lab.pem`. The Private EC2 (Mumbai) uses `new_key.pem`. Key pairs belong to one region, so the Mumbai key must be copied to the VPN EC2 before the second SSH.

1. On **Windows**: copy the Mumbai key to the VPN EC2.
2. On the **VPN EC2**: protect the key.
3. On the **VPN EC2**: SSH to the Private EC2.

```
# 1) Windows cmd (folder that contains both .pem files)
scp -i "ec2-lab.pem" "new_key.pem" ec2-user@3.239.215.44:~

# 2) On the VPN EC2
chmod 400 new_key.pem

# 3) On the VPN EC2
ssh -i new_key.pem ec2-user@10.0.0.164
```

Example output (your numbers and names may look a little different)

```
[ec2-user@ip-192-168-0-44 ~]$ ssh -i new_key.pem ec2-user@10.0.0.164
...
[ec2-user@ip-10-0-0-164 ~]$
```

6.3 Nested SSH confusion explained

Question	Answer
Which machine am I on?	Read the prompt. ip-192-168-0-44 = VPN EC2. ip-10-0-0-164 = Private EC2.
How do I go back one level?	Type exit . You return to the previous machine, not to Windows.
Why can the VPN EC2 reach 10.0.0.164 but Windows cannot?	Windows is not inside the VPN. The VPN EC2 is the tunnel end.
"Permission denied (publickey)"	Wrong or missing key file. Use <code>new_key.pem</code> for the Private EC2, <code>ec2-lab.pem</code> for the VPN EC2.
"Unprotected private key file"	Run <code>chmod 400</code> on the key (Linux). On Windows, remove other users from the file Security tab.

Tip: Cleaner and safer way: `ssh -J ec2-user@3.239.215.44 -i new_key.pem ec2-user@10.0.0.164` (jump host). In real companies, do not leave private keys on servers; use SSH agent forwarding or jump hosts.

7. Real-Life Practical Demo

Ping only proves the road exists. Now we show **real application traffic**: reading and copying company files over the VPN.

Machine	Role in the story
Private EC2 (10.0.0.164)	Company private data server in AWS. Holds the company files.
VPN EC2 (192.168.0.44)	Client / office network side. Reads and copies the files.

7.1 Create dummy company data on the Private EC2

1. SSH to the Private EC2 (see chapter 6).
2. Create the folder **company-data** and a few dummy files (not real data).

```
mkdir ~/company-data
cd ~/company-data
echo "id,name,department" > employees.csv
echo "1,Asha,HR" >> employees.csv
echo "2,Ravi,Finance" >> employees.csv
echo "Q3 sales target: 1000 units" > sales_report.txt
echo "Project Alpha: kickoff next Monday" > project_plan.txt
ls -l
```

7.2 Read the files from the VPN EC2 through the VPN

```
# on the VPN EC2
ssh -i new_key.pem ec2-user@10.0.0.164 "ls -l ~/company-data"
ssh -i new_key.pem ec2-user@10.0.0.164 "cat ~/company-data/employees.csv"
```

Example output (your numbers and names may look a little different)

```
id,name,department
1,Asha,HR
2,Ravi,Finance
```

7.3 Copy the files with scp

```
# on the VPN EC2
mkdir ~/downloaded
scp -i new_key.pem -r ec2-user@10.0.0.164:~/company-data ~/downloaded/
ls -l ~/downloaded/company-data
```

Example output (your numbers and names may look a little different)

```
employees.csv      100%  45    0.2KB/s   00:00
sales_report.txt   100%  29    0.1KB/s   00:00
project_plan.txt   100%  35    0.1KB/s   00:00
```

7.4 "Permission denied" while copying: reason and solution

Message	Reason	Solution
scp: remote open "/home/ec2-user/company-data/xyz": Permission denied	The file or folder was created with sudo , so the owner is root and other users cannot read it.	On the Private EC2: sudo chown -R ec2-user:ec2-user ~/company-data chmod -R u+rwX,go+r ~/company-data
Permission denied (publickey)	Wrong key file used for the login.	Use new_key.pem for the Private EC2.
scp: ... Permission denied (writing to local folder)	No write permission on the destination folder on the VPN EC2.	Copy into your home folder: ~/downloaded

7.5 Prove the data really went through the VPN

```
# Terminal 1 on the VPN EC2: watch the wire
sudo tcpdump -ni any "esp or udp port 4500"

# Terminal 2 on the VPN EC2: copy the files again
scp -i new_key.pem -r ec2-user@10.0.0.164:~/company-data ~/downloaded2/

# After the copy, byte counters have grown
sudo ipsec trafficstatus
```

- In Terminal 1 you see **ESP** or **UDP 4500** packets, not plain SSH packets. This means the data was encrypted.
- The Private EC2 has **no public IP**, so the only possible path is the VPN.
- The **inBytes** / **outBytes** numbers in trafficstatus grow after the copy.

Why this matches real life: in a real company, applications (databases, file shares, ERP) talk to private servers using private IPs. SSH and scp are normal TCP application traffic, just like a database or file-share connection. If scp works across the VPN, other business applications will work too.

8. Important Theory (short and simple)

8.1 Site-to-Site vs Remote-Access VPN

Point	Site-to-Site VPN	Remote-Access (Client) VPN
Joins	Network to network	One person (laptop) to a network
Software on user device	Not needed	VPN client needed
Example	Head office to AWS (this lab)	Employee working from home
Always on?	Yes, 24x7 between gateways	Only when the user connects
AWS tools	VGW / Transit Gateway + CGW	Client VPN endpoint, or OpenVPN server on EC2

8.2 IPsec

IPsec is a set of rules (a protocol suite) that secures IP traffic. It gives **encryption** (nobody can read), **integrity** (nobody can change it) and **authentication** (you know who is on the other side).

8.3 IKE

IKE (Internet Key Exchange) is the "handshake". It works in two phases. **Phase 1**: both sides check identity (PSK) and build a safe control channel. **Phase 2**: they agree on keys for the real data. Our lab used **IKEv1**. IKE uses **UDP 500** (and **UDP 4500** when NAT is in the path).

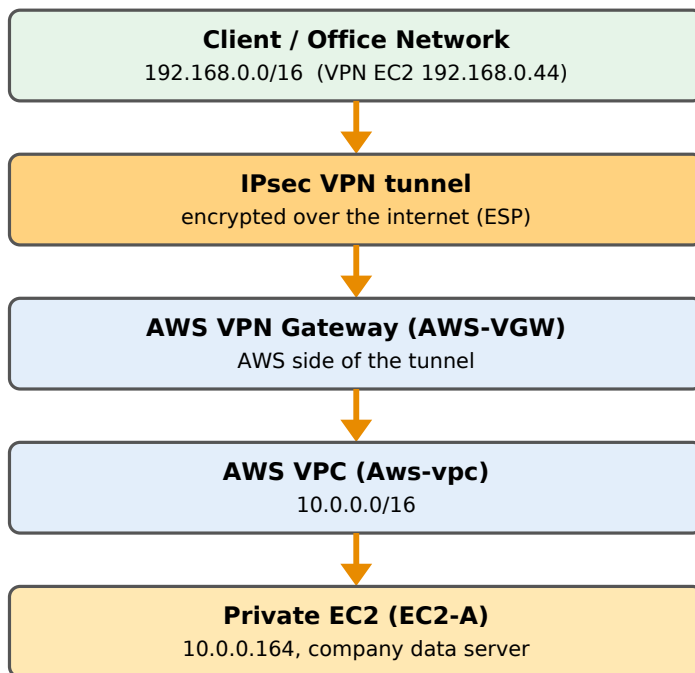
8.4 ESP and encryption

ESP (Encapsulating Security Payload) is the part that carries your data in the tunnel. It **encrypts** the original packet (here with AES-128) and adds a check (SHA-1) so changes are detected. In tunnel mode the whole original packet, including the private IPs, is hidden inside a new packet.

8.5 Other words

Term	Easy explanation
Tunnel vs normal routing	Normal routing sends plain packets. A tunnel wraps and encrypts the packet inside another packet.
VGW (Virtual Private Gateway)	The AWS-managed VPN endpoint attached to one VPC. It gives 2 tunnels automatically.
CGW (Customer Gateway)	An AWS record that describes your VPN device (public IP, ASN).
PSK	The shared secret used to prove identity in Phase 1. Each tunnel has its own PSK.
Static vs dynamic (BGP) routing	Static: you type the networks. Dynamic: the two sides share routes automatically with BGP.
NAT-T	NAT Traversal. Wraps ESP in UDP 4500 so IPsec works when the server sits behind NAT (as EC2 does).
DPD	Dead Peer Detection. Small "are you alive?" checks; restarts the tunnel if the peer is gone.
XFRM	The Linux kernel framework that encrypts and decrypts packets using the keys Libreswan agreed.
Private IP communication	Servers talk with 10.x and 192.168.x addresses. These are not routable on the internet; the VPN makes it possible safely.

9. Final Working Architecture



Traffic flow when the VPN EC2 pings or copies a file from 10.0.0.164

1. The VPN EC2 (192.168.0.44) sends a packet to 10.0.0.164.
2. The XFRM policy says "10.0.0.0/16 must use the tunnel", so the kernel **encrypts** the packet with ESP.
3. The encrypted packet goes out through DC-IGW over the internet to the AWS tunnel IP 13.201.140.220.
4. The **VGW** decrypts it and delivers the original packet inside Aws-vpc.
5. The Private EC2 receives it (the security group allows ICMP / SSH from 192.168.0.0/16).
6. The reply goes back: aws-rt says "192.168.0.0/16 to VGW", the VGW encrypts it and sends it through the tunnel.
7. The VPN EC2 decrypts the reply. The user sees a normal answer, as if both servers were in one building.

10. Commands Cheat Sheet

Command	Purpose	Expected output
<code>ssh -i "ec2-lab.pem" ec2-user@3.239.215.44</code>	Login Windows to VPN EC2	Amazon Linux 2023 banner, prompt ip-192-168-0-44
<code>sudo dnf update -y</code>	Update packages	Complete! (or Nothing to do)
<code>sudo dnf install libreswan -y</code>	Install IPsec software	Installed: libreswan-4.12 ... Complete!
<code>ipsec --version</code>	Check Libreswan	Libreswan 4.12
<code>sudo nano /etc/sysctl.conf</code>	Edit kernel settings	Editor opens
<code>sudo sysctl -p</code>	Apply kernel settings	Prints the 3 values (1, 0, 0)
<code>sudo nano /etc/ipsec.d/aws.conf</code>	Tunnel configuration	Editor opens
<code>sudo nano /etc/ipsec.d/aws.secrets</code>	Store the PSK	Editor opens
<code>sudo chmod 600 /etc/ipsec.d/aws.secrets</code>	Protect the secret file	No output
<code>sudo systemctl enable --now ipsec</code>	Start IPsec now and at boot	Created symlink ...
<code>sudo systemctl status ipsec</code>	Is service running?	active (running)
<code>sudo ipsec status</code>	Tunnel list and states	IPsec SA established
<code>sudo ipsec trafficstatus</code>	Bytes through the tunnel	inBytes=... outBytes=...
<code>sudo ip xfrm state</code>	Kernel SAs (keys, SPI)	src ... dst ... proto esp spi ...
<code>sudo ip xfrm policy</code>	Which traffic is encrypted	src 192.168.0.0/16 dst 10.0.0.0/16 dir out
<code>ping -c 4 10.0.0.164</code>	Test connection	4 received, 0% packet loss
<code>sudo tcpdump -ni any icmp</code>	Watch ping packets	echo request / echo reply lines
<code>sudo tcpdump -ni any "esp or udp port 4500"</code>	Prove encryption	ESP / UDP 4500 packets
<code>sudo iptables -L -n -v</code>	Check Linux firewall	Policy ACCEPT
<code>systemctl status firewalld</code>	Check firewalld	not found or inactive
<code>scp -i "ec2-lab.pem" "new_key.pem" ec2-user@3.239.215.44:~</code>	Copy Mumbai key to VPN EC2	100% transferred
<code>chmod 400 new_key.pem</code>	Protect key file	No output
<code>ssh -i new_key.pem ec2-user@10.0.0.164</code>	Login VPN EC2 to Private EC2	prompt ip-10-0-0-164
<code>scp -i new_key.pem -r ec2-user@10.0.0.164:~/company-data ~/downloaded/</code>	Copy company files	100% for each file
<code>sudo chown -R ec2-user:ec2-user ~/company-data</code>	Fix Permission denied	No output

11. Interview Explanation

Q. Explain this project.

A. I built a Site-to-Site VPN between an AWS network in Mumbai (10.0.0.0/16) and a simulated data centre in N. Virginia (192.168.0.0/16). On AWS I created a Virtual Private Gateway, a Customer Gateway and a VPN connection. In the data centre I used an EC2 server with Libreswan as the office VPN router. After the IPsec tunnel came up, I pinged and SSH-ed to a private EC2 with no public IP, and copied dummy company files with scp over the tunnel.

Q. Why did you use Site-to-Site VPN?

A. The company needs to connect two whole networks securely. A Site-to-Site VPN is encrypted, quick to set up and cheap, and users need no VPN software. Direct Connect is better for very high speed but costs more and takes longer to arrange.

Q. How does traffic flow?

A. The VPN EC2 sends a packet to 10.0.0.164. The kernel XFRM policy encrypts it with ESP. It goes over the internet to the AWS tunnel endpoint, the VGW decrypts it and delivers it in Aws-vpc. The reply returns because aws-rt has the route 192.168.0.0/16 to the VGW.

Q. How did you troubleshoot it?

A. Step by step: first I checked that the tunnel was up (ipsec status, AWS console). Then I tested ping. I checked the destination IP, security groups, route tables, firewall (iptables, firewallD), and used tcpdump and ip xfrm to see if packets entered the tunnel.

Q. What was the issue and how did you solve it?

A. First, ping to 10.0.0.1 failed because that address is the VPC router, not a server. The real private IP was 10.0.0.164. Then ping still failed because the Private EC2 security group did not allow ICMP. I added an ICMP rule from 192.168.0.0/16 and ping worked. A later scp "Permission denied" came from file ownership (created with sudo); I fixed it with chown.

Q. Extra: "Why two tunnels?"

A. AWS gives two tunnels in different zones for high availability. Libreswan can keep only one up at a time, so the second one is a backup.

Q. Extra: "Static or dynamic routing?"

A. I used static routing (192.168.0.0/16). Dynamic routing uses BGP and is better for many networks, but it needs extra routing software.

Q. Extra: "What would you improve for production?"

A. Use an Elastic IP on the VPN server, restrict SSH and ICMP sources, use two VPN servers or a managed firewall for high availability, and use BGP.

Lab finished. Remember to clean up to avoid cost: delete the VPN connection (it is billed per hour), VGW, CGW, and stop or terminate the EC2 servers when you are done.